Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiments 8, 9

Effect of Attention Type and Attention Position in a Pretrained ResNet50

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name : Ashwin S
Reg no : 24011101007
Class : AI DS - A (batch 1)

1. Objective

The objective of this experiment is to understand how different attention mechanisms modify a pretrained convolutional neural network and how the **position at which attention is inserted** influences image classification performance.

A single pretrained architecture,

ResNet50 pretrained on ImageNet

is used throughout the experiment.

This report investigates:

- channel attention;
- spatial attention;
- sequential channel–spatial attention;
- lightweight self-attention;
- different positions for inserting attention within ResNet50;
- single-stage and multi-stage attention;
- attention ordering;
- additional established attention mechanisms; and
- qualitative interpretation using internal attention maps and Grad-CAM.

The experiment emphasizes that the effectiveness of attention depends not only on the attention mechanism itself, but also on the characteristics of the feature representation on which it operates.

2. Dataset

Dataset: TensorFlow Flowers (`tf_flowers`)

The TensorFlow Flowers dataset contains natural RGB flower images belonging to five classes:

`daisy`, `dandelion`, `roses`, `sunflowers`, `tulips`.

The images contain variations in:

- scale;
- viewpoint;
- illumination;
- background clutter;
- flower size;
- color;
- texture; and
- object location.

These variations make the dataset suitable for studying the behavior of attention mechanisms.

In this run, `tfds.load` could not be used on the Kaggle image because a Protobuf version mismatch broke the `tensorflow_datasets` import. The data were therefore read from the official `flower_photos` archive, which is the same source that TFDS downloads for `tf_flowers`. The images and classes are identical, and the labels were mapped to the TFDS index order (dandelion, daisy, tulips, sunflowers, roses). All 3,670 images were used.

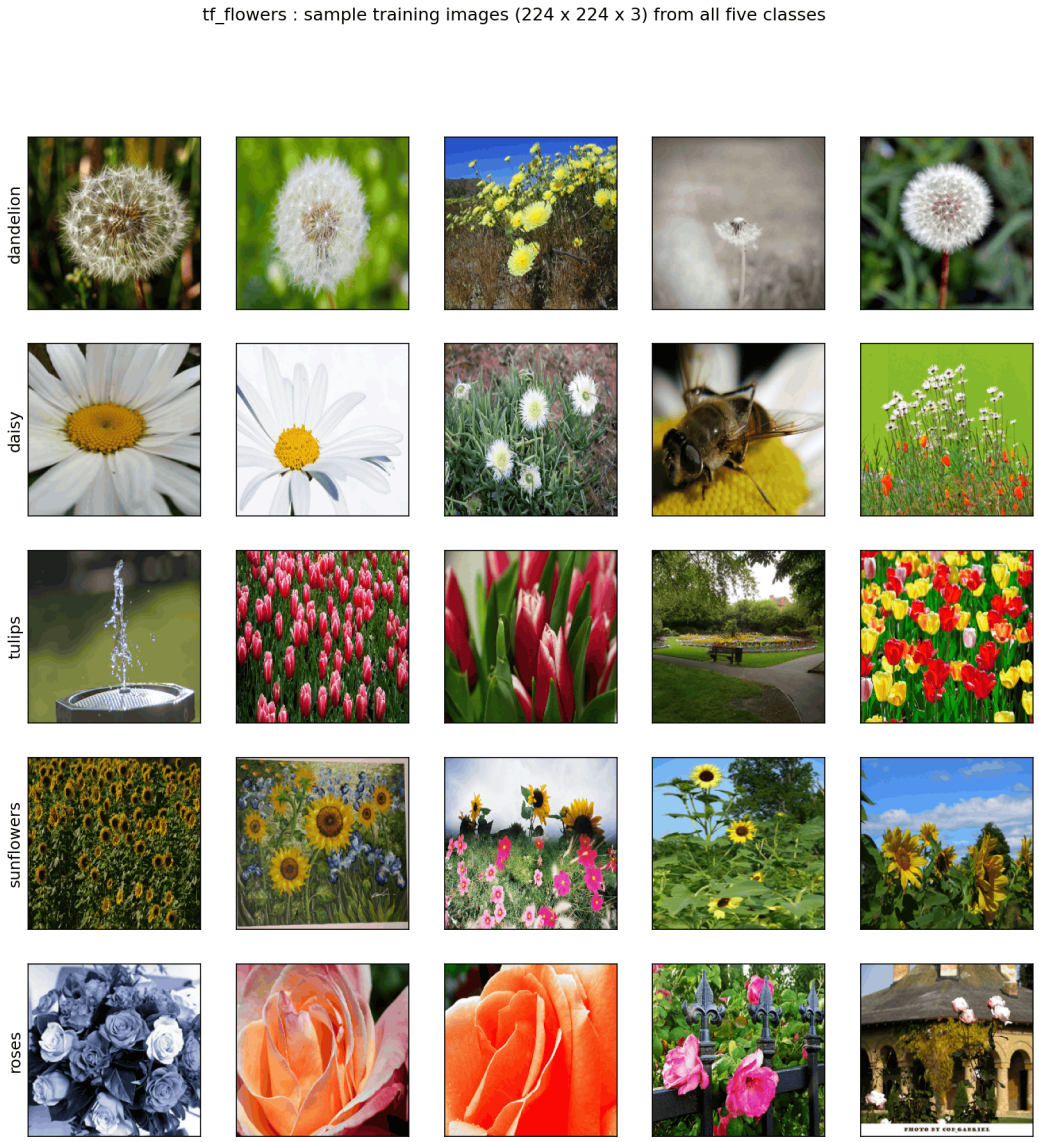


Figure 1: Sample training images from all five classes after resizing to $224 \times 224 \times 3$.

The classes differ a lot in scale and framing, from single close-up blooms to whole fields, and a few labels are clearly noisy (the tulip row contains a water fountain and a park scene with only a distant flowerbed). Such images carry little flower evidence for any model, so they put a ceiling on accuracy that attention alone cannot remove.

2.1 Image Preprocessing

Every input image was resized to

$$224 \times 224 \times 3.$$

The image was preprocessed using the preprocessing procedure expected by the ImageNet-pretrained ResNet50.

The data augmentation used included:

- random horizontal flipping;
- small rotations;
- random zoom;
- random translation; and
- mild contrast variation.

The same augmentation strategy was maintained for all model comparisons.

Each image was resized directly to 224×224 with bilinear interpolation (the aspect ratio is not preserved) and then passed through the ResNet50 "caffe" preprocessing (RGB→BGR and ImageNet mean subtraction). I checked that this gives exactly the same output as `keras.applications.resnet50.preprocess_input`. The augmentation used for every model was a random horizontal flip, a rotation of up to $\pm 18°$, zoom and translation of up to $\pm 10\%$, and a contrast change of up to $\pm 10\%$. It was applied only to training images and re-created with the same seed (42) at the start of every run.

2.2 Dataset Split

The same training, validation, and testing split was used for every experiment:

$$\boxed{70\% : 15\% : 15\%}$$

for

$$\text{Training} : \text{Validation} : \text{Testing}.$$

The exact same samples were retained in each subset for all ablation experiments.

This was necessary to ensure that differences in performance are caused by changes in the model rather than changes in the dataset split.

The split was stratified by class with a fixed seed (42). The selected indices (with source file names) were saved once and reused by all seventeen trained models, giving the following subsets:

Class	**Train**	**Validation**	**Test**	**Total**
dandelion	629	134	135	898
daisy	443	95	95	633
tulips	559	120	120	799
sunflowers	489	105	105	699
roses	449	96	96	641
Total	**2569**	**550**	**551**	**3670**

3. Pretrained Architecture: ResNet50

ResNet50 pretrained using ImageNet is used as the **only backbone architecture** in this experiment.

For an input image

$$I \in \mathbb{R}^{224 \times 224 \times 3},$$

the major ResNet50 stages produce approximately the following feature dimensions:

ResNet50 Stage	**Spatial Resolution**	**Channels**
Stem + MaxPool	56×56	64
conv2_x	56×56	256
conv3_x	28×28	512
conv4_x	14×14	1024
conv5_x	7×7	2048

As the network becomes deeper:

$$\boxed{\text{Spatial Resolution} \downarrow}$$

while

$$\boxed{\text{Feature Channels} \uparrow}$$

and the representation generally progresses from low-level visual information toward higher-level semantic information.

Keras' built-in ResNet50 does not allow a layer to be inserted between two stages, so the network was rebuilt layer by layer with the same layer names and the ImageNet weights were copied into it. The rebuilt backbone reproduced the reference model exactly (maximum absolute output difference 0.000), has 23,587,712 parameters without the classification top, and gave the stage dimensions listed above.

4. Understanding ResNet50 Feature Hierarchy

The approximate interpretation of different stages is:

- **Early features:** edges, colors, orientations, boundaries, and simple textures.
- **Intermediate features:** local patterns, repeated structures, flower parts, petal arrangements, and shape information.
- **Late features:** high-level object structures and class-discriminative semantic information.

Therefore, the same attention mechanism may produce different behavior when inserted at different locations.

5. Why Attention Position Matters

Attention is not independent of the features on which it operates.
Suppose an attention module receives

$$F \in \mathbb{R}^{H \times W \times C}.$$

The values of

$$H, \qquad W, \qquad C$$

and the semantic meaning of the feature tensor change significantly with network depth.
Therefore:

- **early-stage attention** operates on high-resolution, low-level visual information;
- **middle-stage attention** operates on intermediate features containing both spatial and semantic information; and
- **late-stage attention** operates on low-resolution, high-level semantic representations.

This report therefore investigates both:

$$\boxed{\text{Which attention mechanism should be used?}}$$

and

$$\boxed{\text{Where should the attention mechanism be inserted?}}$$

6. Candidate Attention Positions

Four candidate attention positions in ResNet50 are defined.

$$P_1: \text{ after conv2_x},$$

$$P_2: \text{ after conv3_x},$$

$$P_3: \text{ after conv4_x},$$

$$P_4: \text{ after conv5_x}.$$

The corresponding feature dimensions are:

Position	**Location**	**Resolution**	**Channels**
P_1	After conv2_x	56×56	256
P_2	After conv3_x	28×28	512
P_3	After conv4_x	14×14	1024
P_4	After conv5_x	7×7	2048

The principal laboratory exercise concentrates on

$$\boxed{P_2, \quad P_3, \quad P_4}$$

because P_1 contains comparatively high-resolution features and may substantially increase the computational cost of some attention mechanisms.

P_1 was still evaluated once (model B1 in Main Experiment 2), because the channel–spatial module is cheap there (8,563 parameters). Self-attention was not placed at P_1, for the cost reason above.

7. Baseline Model

The baseline is the pretrained ResNet50 without any additional attention mechanism.



The baseline establishes the reference performance against which the attention-based models are compared.

In the implementation, the dropout rate was 0.3. The dense layer produces logits that go through a separate softmax, so the pre-softmax class score y^c is available for Grad-CAM. The baseline has 23,597,957 parameters in total.

8. Channel Attention

Consider a feature tensor

$$F \in \mathbb{R}^{H \times W \times C}.$$

Channel attention determines the relative importance of different feature channels.

Global average pooling is first used to generate a channel descriptor:

$$z_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} F(i, j, c).$$

Thus,

$$z \in \mathbb{R}^C.$$

The channel descriptor is transformed using two learnable transformations:

$$M_c(F) = \sigma\left(W_2 \delta\left(W_1 z\right)\right),$$

where:

- $\delta(\cdot)$ denotes a nonlinear activation such as ReLU;
- $\sigma(\cdot)$ denotes the sigmoid activation;
- W_1 reduces the channel representation; and
- W_2 restores the channel dimension.

The refined feature tensor is:

$$F_c = F \odot M_c(F).$$

Channel attention attempts to answer:

$$\boxed{\text{Which feature channels are important?}}$$

A reduction ratio of

$$r = 16$$

was used as the default setting throughout the laboratory experiment.

The reduction ratio was not treated as a separate mandatory ablation.

$r = 16$ was used in every model. With this setting the module is exactly the Squeeze-and-Excitation block, so the "Channel / SE" entry of the additional comparison reuses this model (A1) instead of training an identical copy. At P_3 it adds 132,160 parameters.

9. Spatial Attention

Spatial attention determines the spatial regions that should receive greater importance.
For the input feature tensor

$$F \in \mathbb{R}^{H\times W\times C},$$

compute average pooling across channels:

$$F_{\text{avg}} = \text{Mean}_c(F),$$

and maximum pooling across channels:

$$F_{\max} = \text{Max}_c(F).$$

Both tensors have dimensions

$$H \times W \times 1.$$

Concatenate them:

$$F_s = [F_{\text{avg}} \| F_{\max}].$$

The spatial attention map is then computed as

$$M_s(F) = \sigma\left(\text{Conv}_{7\times 7}(F_s)\right).$$

Therefore,

$$M_s(F) \in \mathbb{R}^{H\times W\times 1}.$$

The spatially refined feature is

$$F'_s = F \odot M_s(F).$$

Spatial attention answers:

$$\boxed{\text{Which spatial locations are important?}}$$

The 7×7 kernel was used for every model except Main Experiment 5. The module adds only 99 parameters ($2\times 7\times 7$ weights plus a bias), whatever the position.

10. Sequential Channel–Spatial Attention

Channel and spatial attention can be sequentially applied.
First,

$$F_c = M_c(F) \odot F.$$

Then,

$$F_{cs} = M_s(F_c) \odot F_c.$$

The overall structure becomes:

$$\boxed{F \to \text{Channel Attention} \to \text{Spatial Attention} \to F_{cs}}$$

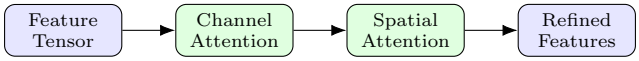
This arrangement first determines

"what is important"

and then determines

"where it is important."

A simplified diagram is shown below.



11. Self-Attention

Self-attention allows spatial locations to interact directly with other spatial locations.
For

$$F \in \mathbb{R}^{H \times W \times C},$$

flatten the spatial dimensions:

$$X \in \mathbb{R}^{N \times C},$$

where

$$N = HW.$$

Generate the query, key, and value matrices:

$$Q = XW_Q,$$

$$K = XW_K,$$

and

$$V = XW_V.$$

The attention matrix is

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right),$$

where d is the dimension of the query/key representation.
The attended output is

$$Y = AV.$$

After reshaping the output back to the original spatial structure, a residual formulation is used:

$$F_{\text{SA}} = F + \gamma\,\text{reshape}(Y),$$

where γ is a learnable scaling parameter.
Self-attention attempts to answer:

$$\boxed{\text{Which spatial features are related to other spatial features?}}$$

Since the attention matrix has dimension approximately

$$N \times N,$$

its computational requirements increase rapidly as

$$N = HW$$

increases.
Therefore, self-attention was preferably introduced at

$$P_3$$

or

$$P_4.$$

In the implementation, W_Q, W_K and W_V are 1×1 convolutions. The query and key use $d = C/8$ (128 at P_3), the value keeps all C channels, and γ starts at zero so that the module begins as an identity mapping. The layer also returns A, but only for visualisation. At P_3 the module adds 1,312,001 parameters. The size of A at each position shows why P_3/P_4 are preferred:

Position	**Resolution**	$N = HW$	**Size of** A	**Memory / image (float32)**
P_1	56×56	3136	3136×3136	37.52 MB
P_2	28×28	784	784×784	2.34 MB
P_3	14×14	196	196×196	0.147 MB
P_4	7×7	49	49×49	0.009 MB

12. Inserting Attention into ResNet50

For any selected stage P_k, the network structure can be written as

$$\text{Previous ResNet Stage} \rightarrow \text{Attention} \rightarrow \text{Next ResNet Stage.}$$

For example, attention inserted after conv4_x gives:

$$\boxed{\text{conv4_x} \rightarrow \text{Attention} \rightarrow \text{conv5_x}}$$

For channel–spatial attention:

$$\boxed{\text{conv4_x} \rightarrow \text{Channel Attention} \rightarrow \text{Spatial Attention} \rightarrow \text{conv5_x}}$$

The feature dimensions immediately before and after the attention block were recorded for every model.

Attention does not unintentionally modify the spatial or channel dimensions unless the selected mechanism explicitly requires such a transformation.

The dimensions recorded immediately before and after every attention block are given below. No module changed the shape of the tensor it received.

Model	**Attention**	**Position**	**Placement**	**Input** (H, W, C)	**Output** (H, W, C)
A1, A2, A3, A4, A5	Channel / Spatial / C→S / SA / C→S→SA	P_3	conv4_x → Attention → conv5_x	(14, 14, 1024)	(14, 14, 1024)
B1	Channel→Spatial	P_1	conv2_x → Attention → conv3_x	(56, 56, 256)	(56, 56, 256)
B2	Channel→Spatial	P_2	conv3_x → Attention → conv4_x	(28, 28, 512)	(28, 28, 512)
B4	Channel→Spatial	P_4	conv5_x → Attention → GAP	(7, 7, 2048)	(7, 7, 2048)
$P_3 + P_4$	Channel→Spatial	P_3, P_4	both placements above	(14,14,1024), (7,7,2048)	unchanged
$P_2 + P_3 + P_4$	Channel→Spatial	P_2, P_3, P_4	all three placements	(28,28,512), (14,14,1024), (7,7,2048)	unchanged
O_SC, K3, K5	S→C, C→S ($k = 3$), C→S ($k = 5$)	P_3	conv4_x → Attention → conv5_x	(14, 14, 1024)	(14, 14, 1024)
ECA, CBAM, COORD	ECA / CBAM / Coordinate	P_3	conv4_x → Attention → conv5_x	(14, 14, 1024)	(14, 14, 1024)

13. Main Experiment 1: Attention-Type Ablation

The first experiment studies the effect of different types of attention.

The attention position was fixed at

$$\boxed{P_3 = \text{after conv4_x}}$$

for all models.

The following were compared:

Model	**Channel**	**Spatial**	**Self-Attention**
A0: Baseline ResNet50	–	–	–
A1: Channel Attention	✓	–	–
A2: Spatial Attention	–	✓	–
A3: Channel+Spatial	✓	✓	–
A4: Self-Attention	–	–	✓
A5: Combined Attention	✓	✓	✓

This experiment investigates:

How does attention type affect model performance?

The following were kept identical:

- dataset split;
- preprocessing;
- augmentation;
- optimizer;
- learning rate schedule;
- epochs;
- batch size;
- backbone;
- classifier;
- attention position; and
- evaluation procedure.

13.1 Results

All six models were trained and evaluated under the protocol of Section 19 (Transfer-Learning Strategy).

Model	Accuracy	Precision	Recall	Macro F1	Attention Params	Inference (ms/img)	Best Val Acc (epoch)
A0: Baseline ResNet50	0.9292	0.9282	0.9274	0.9277	0	9.38	0.9327 (17)
A1: Channel Attention	0.9256	0.9262	0.9223	0.9237	132,160	9.70	0.9364 (20)
A2: Spatial Attention	0.9310	0.9336	0.9267	0.9295	99	9.72	0.9291 (17)
A3: Channel+Spatial	**0.9328**	0.9326	**0.9296**	**0.9309**	132,259	9.81	0.9400 (14)
A4: Self-Attention	0.9002	0.8999	0.8972	0.8981	1,312,001	9.89	0.9382 (4)
A5: Combined Attention	**0.9328**	**0.9345**	0.9282	0.9308	1,444,260	9.87	0.9400 (12)

A3 and A5 tie for the best accuracy at 0.9328, which is 514 of 551 test images correct against 512 for the baseline. Spatial attention alone is one image ahead of the baseline, and channel attention alone is two images behind it. So the first four rows sit within a band of about four images, and with a single seed I would not read a strong ranking into that band.

Self-attention is the one clear outlier, 16 images below the baseline. Its selected checkpoint came from epoch 4, which is still in the frozen-backbone phase: validation accuracy peaked very early (0.9382) and never beat that during fine-tuning, so the test model never benefited from conv5_x fine-tuning. A5 adds the same self-attention block but on top of channel→spatial features, and it did not show this early peak. Training curves, bar graphs and confusion matrices for these models are analysed in Sections 22 and 23.

14. Main Experiment 2: Attention-Position Ablation

The second experiment investigates the influence of the position of attention.

The attention type was fixed as:

Channel Attention → Spatial Attention

and the same module was inserted at different stages.

Experiment	Attention Position	Resolution	Channels
B1	After conv2_x	56×56	256
B2	After conv3_x	28×28	512
B3	After conv4_x	14×14	1024
B4	After conv5_x	7×7	2048

The results shed light on whether attention is more beneficial for:

- high-resolution low-level features;
- intermediate representations; or
- low-resolution semantic features.

The principal question is:

How does the location of attention influence performance?

14.1 Results

B3 is the same configuration as A3 (Channel→Spatial at P_3, $k = 7$), so that trained model was reused rather than retrained.

Exp.	Position	Resolution	Channels	Accuracy	Precision	Recall	Macro F1	Attention Params	Inference (ms/img)
A0 (reference)	–	–	–	0.9292	0.9282	0.9274	0.9277	0	9.38
B1	After conv2_x (P_1)	56×56	256	0.8984	0.8971	0.8983	0.8970	8,563	9.62
B2	After conv3_x (P_2)	28×28	512	0.9238	0.9222	0.9223	0.9221	33,411	9.63
B3	After conv4_x (P_3)	14×14	1024	**0.9328**	**0.9326**	**0.9296**	**0.9309**	132,259	9.81
B4	After conv5_x (P_4)	7×7	2048	0.9256	0.9276	0.9225	0.9246	526,563	9.79

Performance rises from P_1 to a peak at P_3 and then falls again at P_4. Only the P_3 placement beats the baseline. The early placements suffer most, and the reason is probably the training setup: during phase 1 every ResNet stage after the module is frozen, so conv3_x–conv5_x keep receiving re-weighted features they were never trained on, and only conv5_x is later allowed to adapt. B2 shows this clearly, with only 0.6291 validation accuracy after its first epoch against about 0.86 for the baseline.

At P_4 the module sits right before global pooling and has 527K trainable parameters. It fit the training set quickly (training accuracy 0.983 already at epoch 10) while its validation loss drifted up to 0.36, which looks like overfitting rather than a lack of capacity. The spatial maps at the different positions are compared in Section 27.

15. Main Experiment 3: Multiple Attention Positions

Attention may also be inserted at more than one ResNet50 stage.

The following were compared:

$$\text{Attention only at } P_3,$$

$$\text{Attention only at } P_4,$$

and

$$\text{Attention at } P_3 + P_4.$$

As an extended experiment, the following was also evaluated:

$$P_2 + P_3 + P_4.$$

The purpose is to investigate whether applying attention repeatedly provides sufficient performance improvement to justify the additional:

- parameters;
- computation;
- memory;
- training time; and
- inference time.

The results below answer:

Is attention at more locations necessarily more effective?

15.1 Results

"Only P_3" reuses A3 and "only P_4" reuses B4. The two multi-position models were trained separately. Computation was measured as GFLOPs per image with the TensorFlow profiler, memory as the peak GPU memory during training, and training time as the wall-clock time for all 20 epochs.

Configuration	Accuracy	Precision	Recall	Macro F1	Attention Params	GFLOPs	Train Time (s)	Inference (ms/img)
No attention (reference)	0.9292	0.9282	0.9274	0.9277	0	7.7512	453.3	9.38
Only P_3	**0.9328**	**0.9326**	**0.9296**	**0.9309**	132,259	7.7523	461.2	9.81
Only P_4	0.9256	0.9276	0.9225	0.9246	526,563	7.7527	446.4	9.80
$P_3 + P_4$	0.9111	0.9129	0.9088	0.9098	658,822	7.7538	456.8	9.64
$P_2 + P_3 + P_4$ (extended)	0.9129	0.9088	0.9116	0.9097	692,233	7.7556	469.5	9.74

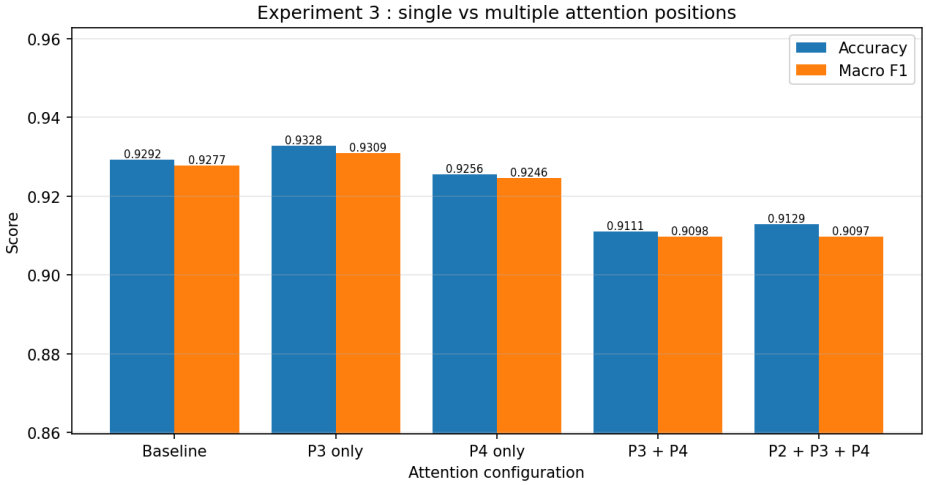


Figure 2: Experiment 3: accuracy and macro F1 for single and multiple attention positions.

A single module at P_3 is the best configuration, and every additional placement makes things worse: $P_3 + P_4$ and $P_2 + P_3 + P_4$ both fall about 1.6–1.8 points below even the plain baseline. Stacking modules gives a frozen backbone more freshly initialised layers to adapt to at once, and the $P_3 + P_4$ model ended with the highest validation loss of all runs (0.536) at a training loss of 0.010, a clear overfitting signature.

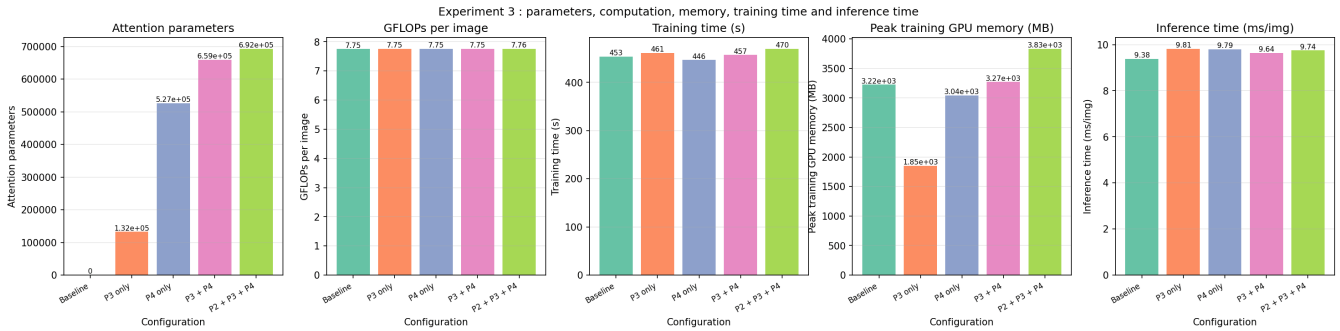


Figure 3: Experiment 3: attention parameters, GFLOPs, training time, peak training GPU memory and inference time.

Attention parameters grow from 132K to 692K, almost entirely because of the 2048-channel module at P_4, yet GFLOPs barely move (7.7512 to 7.7556) and training and inference times stay within a few percent. Peak memory rises with the number of positions (1,849 MB for P_3 only to 3,831 MB for three positions). The baseline's 3,224 MB, however, shows that this number also reflects how TensorFlow's allocator happened to reserve memory, so I treat it as rough. The extra cost is small, but it buys no accuracy, so more positions are not worth it here.

16. Main Experiment 4: Attention Order

The sequence in which channel and spatial attention are applied may influence the resulting features.
The following were compared:

$$\boxed{\text{Channel} \rightarrow \text{Spatial}}$$

against

$$\boxed{\text{Spatial} \rightarrow \text{Channel}}$$

while maintaining the same insertion position.
For example, at P_3:

$$F \rightarrow M_c \rightarrow M_s$$

was compared with

$$F \to M_s \to M_c.$$

The results determine whether selecting informative feature channels before selecting spatial regions produces behavior different from selecting spatial regions first.

16.1 Results

The Channel→Spatial model is A3 (reused). Spatial→Channel was trained with the identical module, position, kernel and protocol.

Order at P_3	Accuracy	Precision	Recall	Macro F1	Attention Params	Inference (ms/img)	Best Val Acc (epoch)	Final Val Loss
No attention (reference)	0.9292	0.9282	0.9274	0.9277	0	9.38	0.9327 (17)	0.3087
Channel→Spatial ($F \to M_c \to M_s$)	**0.9328**	**0.9326**	0.9296	**0.9309**	132,259	9.81	0.9400 (14)	0.3311
Spatial→Channel ($F \to M_s \to M_c$)	0.9310	0.9304	0.9296	0.9300	132,259	9.89	0.9255 (15)	0.4183

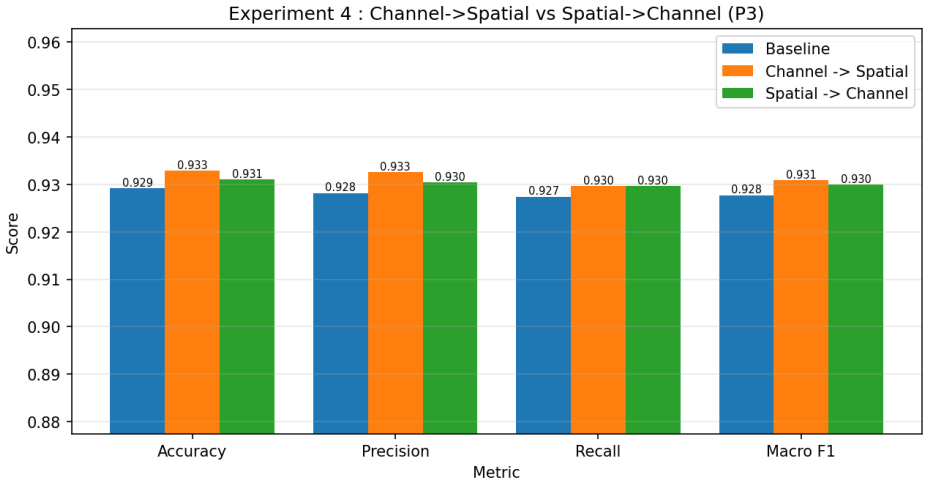


Figure 4: Experiment 4: accuracy, precision, recall and macro F1 for the two attention orders.

The two orders end up practically tied: Channel→Spatial is ahead by one test image in accuracy, and recall is identical (0.9296). On final test numbers alone the order does not matter in this setup, since the gap is smaller than what a different seed could produce.

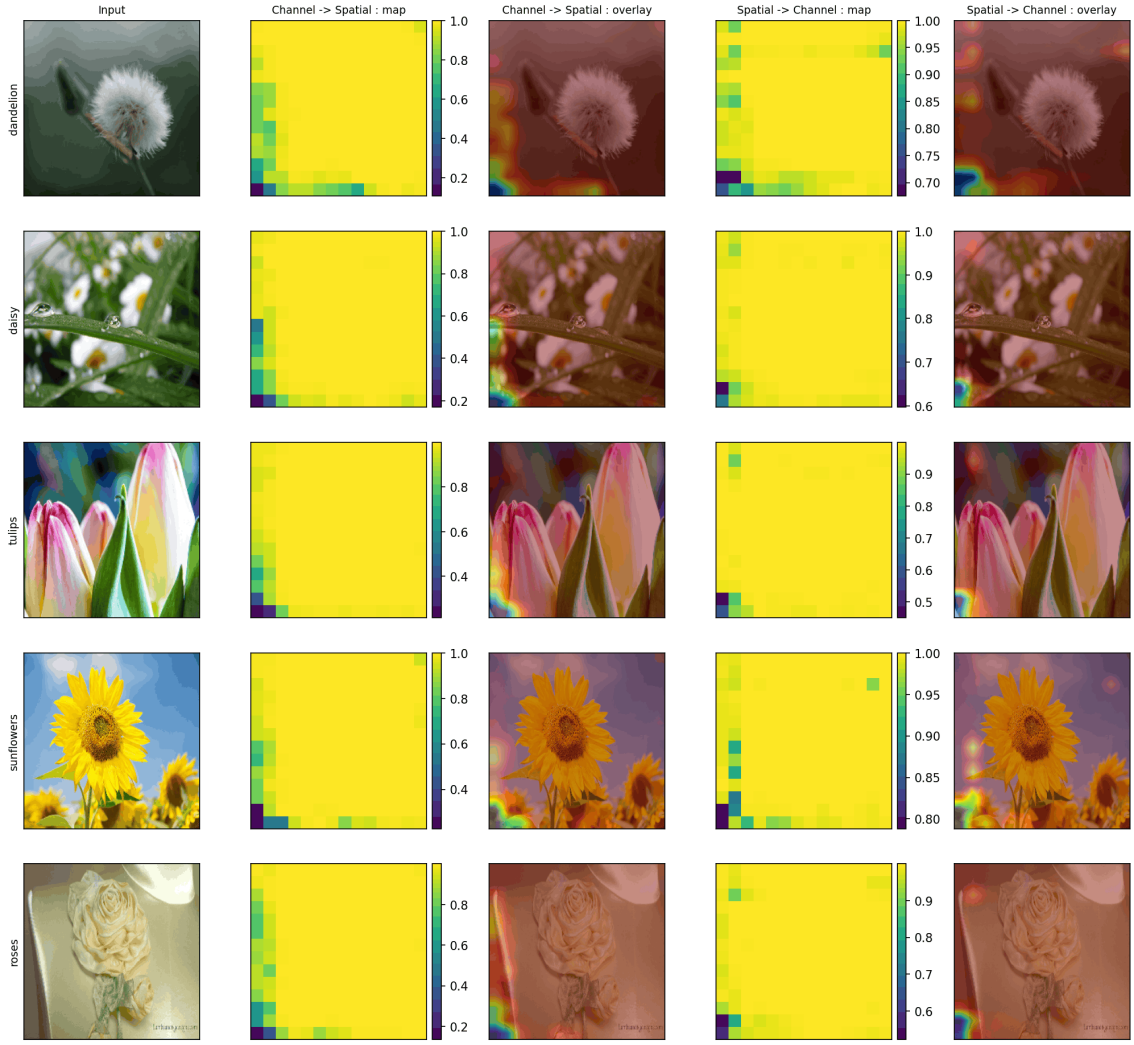


Figure 5: Experiment 4: spatial-attention maps and overlays for Channel→Spatial and Spatial→Channel.

Both orders learn border-shaped maps that stay close to 1 inside the image, but the Spatial→Channel maps have a much narrower range: their lowest values are around 0.5–0.8, against about 0.15–0.2 for Channel→Spatial. When the spatial gate acts first, on raw conv4_x features, it suppresses far less. That order also trained more awkwardly, with 0.18 validation accuracy after epoch 1 (close to chance) and a higher final validation loss (0.418 vs 0.331). So order changed the training behaviour more than the final score.

17. Main Experiment 5: Spatial Attention Kernel Size

The receptive field used to generate the spatial attention map may influence the result.
The following were evaluated:

$$k \in \{3, 5, 7\}.$$

The same attention position and training protocol were used.

Spatial Kernel	Accuracy	Macro F1	Inference Time (ms/img)	Attention Params	Best Val Acc (epoch)
3×3	**0.9328**	**0.9309**	9.63	132,179	0.9327 (19)
5×5	0.9274	0.9259	9.73	132,211	0.9273 (17)
7×7	**0.9328**	**0.9309**	9.81	132,259	0.9400 (14)

The results below address whether a larger spatial kernel provides a useful increase in contextual information.

All three runs use Channel→Spatial at P_3; the 7×7 run is A3 (reused). The 3×3 and 7×7 models reached exactly the same test accuracy, precision, recall and macro F1, even though they selected different checkpoints, while 5×5 was three images behind.

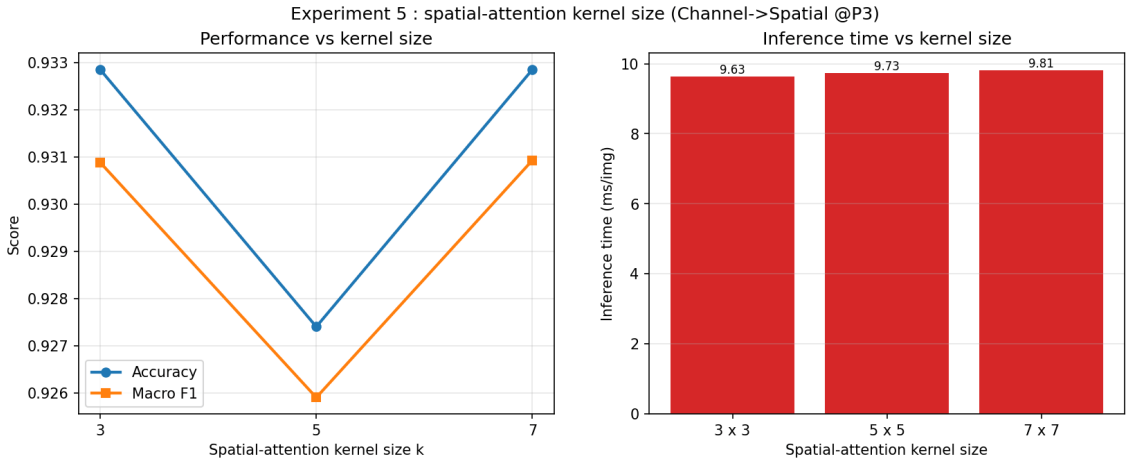


Figure 6: Experiment 5: performance and inference time against spatial-attention kernel size.

Accuracy is V-shaped (0.9328, 0.9274, 0.9328) rather than increasing with k, while inference time creeps up from 9.63 to 9.81 ms. A larger receptive field for the spatial gate does not buy accuracy here, so 3×3 gives the same result as 7×7 for slightly less time and 80 fewer parameters.

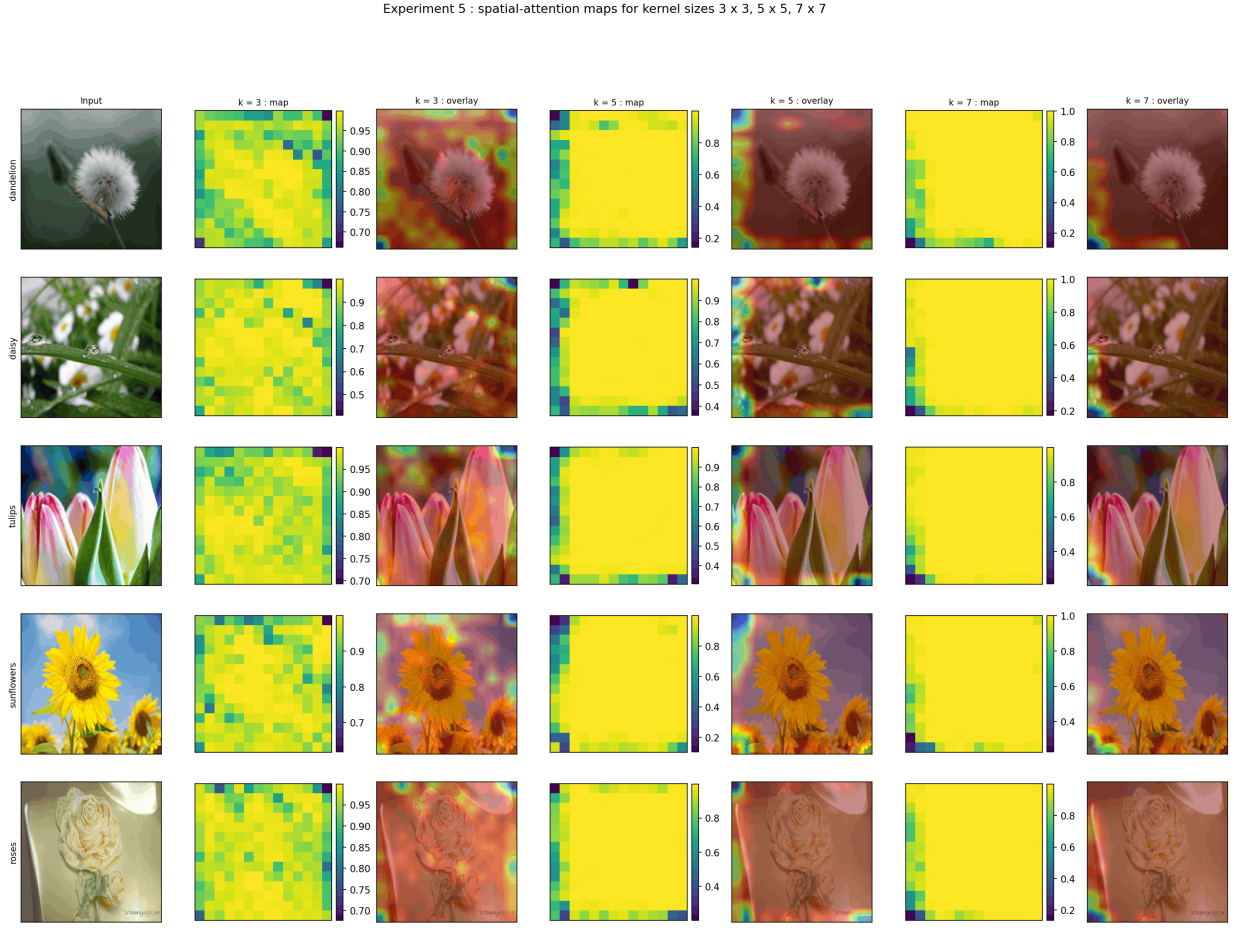


Figure 7: Experiment 5: spatial-attention maps and overlays for $k = 3$, 5 and 7.

The kernel size changes the texture of the map more than its meaning. With $k = 3$ the low values are scattered as small patches through the interior (range about 0.65–1). With $k = 5$ they form a frame around all borders, and with $k = 7$ they are smooth and limited to the left and bottom edges. A larger kernel gives a smoother, more border-driven map, but none of them outlines the flower, which explains why accuracy does not follow k.

18. Additional Task: Comparison of Different Attention Mechanisms

As an additional exploration task, the attention mechanisms implemented in the laboratory are compared with established attention approaches.

At least

$$\boxed{\text{two additional attention mechanisms}}$$

were selected from the following:

- Squeeze-and-Excitation (SE) Attention;
- Efficient Channel Attention (ECA);
- Convolutional Block Attention Module (CBAM);
- Coordinate Attention; and
- lightweight Self-Attention.

The pretrained backbone remains:

$$\boxed{\text{ResNet50}}$$

for all comparisons.

To ensure a fair comparison, the additional attention mechanisms were inserted at the same position:

$$\boxed{P_3 = \text{after conv4_x}}$$

Five mechanisms from this list were compared at P_3: SE, ECA, CBAM, Coordinate Attention and the lightweight self-attention. SE is identical to the channel attention of Section 8 (with $r = 16$), so model A1 serves as the SE entry. The implementation details follow the original papers:

- **ECA:** adaptive 1D kernel ($k = 5$ for 1024 channels).
- **CBAM:** $r = 16$ and a 7×7 spatial kernel, with no BatchNorm in the spatial branch so that it matches the $\sigma(\text{Conv})$ formula of Section 9.
- **Coordinate Attention:** reduction 32, h-swish activation and BatchNorm.

18.1 Comparison

Attention	Accuracy	Precision	Recall	Macro F1	Total Parameters	Attention Params	Inference Time (ms/img)
No Attention	0.9292	0.9282	0.9274	0.9277	23,597,957	0	9.38
Channel / SE	0.9256	0.9262	0.9223	0.9237	23,730,117	132,160	9.70
Spatial Attention	**0.9310**	**0.9336**	0.9267	**0.9295**	23,598,056	99	9.72
ECA	0.9238	0.9256	0.9187	0.9214	23,597,962	5	9.63
CBAM	0.9201	0.9199	0.9155	0.9168	23,730,216	132,259	9.80
Self-Attention	0.9002	0.8999	0.8972	0.8981	24,909,958	1,312,001	9.89
Additional Mechanism: Coordinate Attention	**0.9310**	0.9300	**0.9285**	0.9290	23,698,469	100,512	9.68

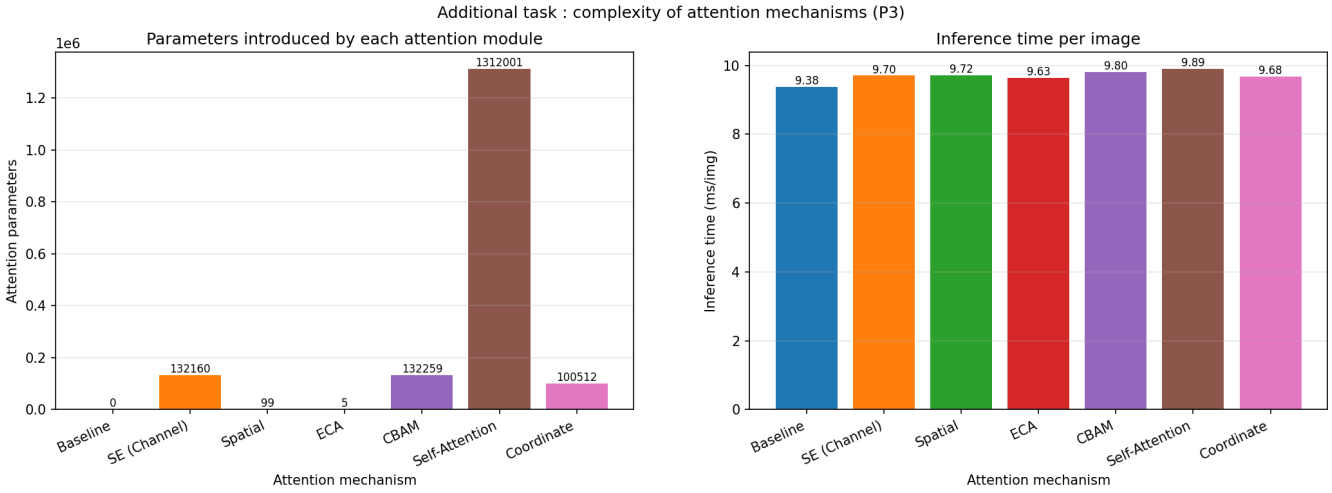


Figure 8: Additional task: parameters introduced by each attention module and inference time per image.

The added parameters span five orders of magnitude, from 5 (ECA) to 1.31M (self-attention), but inference time only spans 9.38–9.89 ms. Apart from self-attention being the slowest, latency hardly tracks parameter count, because

the ResNet50 backbone dominates the runtime. Coordinate attention, with fewer parameters than SE or CBAM, matches the best accuracy in this group (0.9310).

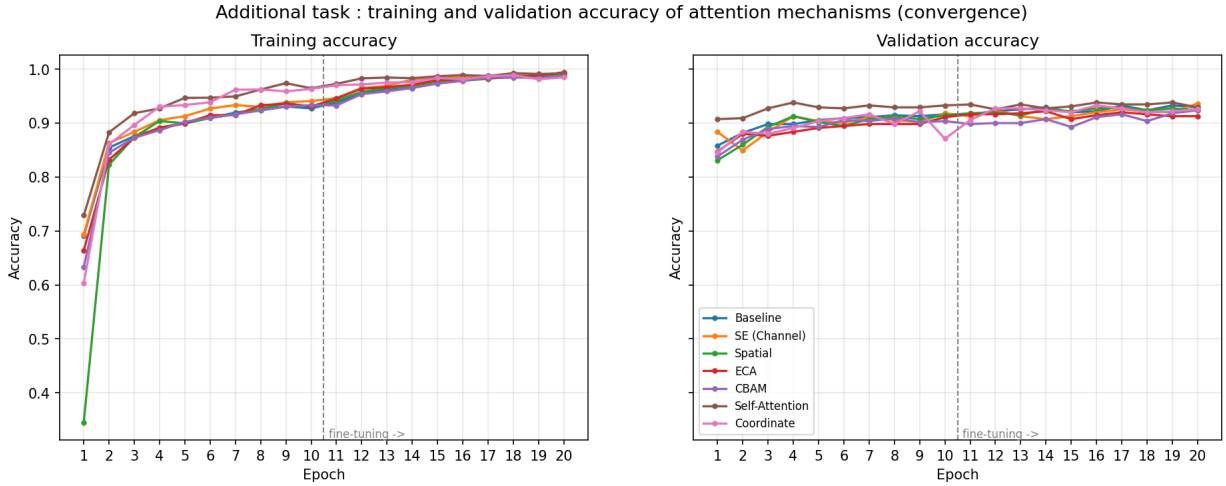


Figure 9: Additional task: training and validation accuracy of the compared mechanisms over 20 epochs.

All models end with training accuracy around 0.985–0.995 while validation stays near 0.91–0.94, so they overfit to a similar degree; no mechanism closes the gap. Self-attention learns fastest in phase 1 (0.907 validation accuracy after one epoch), CBAM climbs the slowest, and Coordinate attention has a visible wobble at epoch 10 (validation accuracy 0.871, loss 0.554) before fine-tuning brings it back to 0.933.

18.2 Discussion of Attention-Mechanism Comparison

1. **How does classification accuracy vary across attention mechanisms?** It ranges from 0.9002 (self-attention) to 0.9310 (spatial and Coordinate), and only those two beat the baseline (0.9292), each by one image.
2. **How does macro F1-score vary across attention mechanisms?** It follows accuracy closely (0.8981 to 0.9295), with a gap of at most 0.005 to accuracy, so no mechanism gains on one class while losing heavily on another.
3. **How many parameters are introduced by each attention module?** ECA adds 5, spatial 99, Coordinate 100,512, SE 132,160, CBAM 132,259 and self-attention 1,312,001.
4. **How does each attention mechanism influence inference time?** Each module adds roughly 0.25–0.5 ms to the baseline's 9.38 ms (3–5%). ECA is the cheapest (9.63 ms) and self-attention the most expensive (9.89 ms).
5. **Does attention improve the convergence behavior?** Only self-attention converged noticeably faster in phase 1. None of them reduced the final train–validation gap, and Coordinate attention was less stable around the switch to fine-tuning.
6. **Does attention produce more focused feature localization?** Not consistently. In Figure 27, CBAM is the most compact on the daisy image and self-attention the most spread out, but on the other images all maps are close to the baseline.
7. **Does an increase in attention complexity always improve classification performance?** No. The most complex module (self-attention) gave the lowest accuracy, and the 99-parameter spatial module was among the best.
8. **Does a lightweight attention mechanism provide comparable performance with a computationally heavier mechanism?** Yes. ECA with 5 parameters is within one image of SE (132K), and spatial attention with 99 parameters beats both SE and CBAM.

19. Transfer-Learning Strategy

ImageNet pretrained weights were used.

The training strategy adopted was:

1. Load ResNet50 with ImageNet pretrained weights.
2. Remove the original ImageNet classification layer.
3. Insert the selected attention module.
4. Add Global Average Pooling.
5. Add Dropout.
6. Add a Dense classification layer containing five output neurons.

7. Initially freeze the pretrained ResNet50 layers.
8. Train the newly introduced attention and classification layers.
9. Unfreeze the final ResNet50 stage.
10. Fine-tune the network using a smaller learning rate.

The initial learning rate used for the newly introduced layers was

$$10^{-3}.$$

During fine-tuning, a smaller learning rate of

$$10^{-5}$$

was used.

The exact values selected remained identical for all controlled comparisons.

The values used for every model were as follows.

- **Optimiser and batch size:** Adam with batch size 32.
- **Phase 1:** 10 epochs at 10^{-3} with the whole ResNet50 frozen, so only the attention module and classifier were trained (for example, 10,245 parameters for the baseline).
- **Phase 2:** 10 epochs at 10^{-5} with only the convolution layers of conv5_x unfrozen. The BatchNorm layers were kept frozen in inference mode, which gives about 15.0M trainable parameters.
- **Schedule:** the learning rate was constant within each phase and there was no early stopping, so every model saw exactly 20 epochs.
- **Checkpoint for testing:** the weights from the epoch with the highest validation accuracy over all 20 epochs were restored before testing, instead of the last epoch.
- **Runs:** each configuration was trained once with seed 42 on a Kaggle Tesla T4. GPU operation determinism was not forced, so a rerun can differ slightly.

20. Loss Function

For the five-class classification problem, categorical cross-entropy was used.

For N samples:

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{5} y_{i,c}\log \widehat{y}_{i,c},$$

where:

$$y_{i,c}$$

is the ground-truth label and

$$\widehat{y}_{i,c}$$

is the predicted probability.

The same loss formulation was used for every attention comparison.

Integer labels were converted to one-hot vectors, and Keras' `categorical_crossentropy` was applied to the softmax output of every model.

21. Main Result Table

All metrics are computed on the 551 held-out test images. Precision, recall and F1 are macro-averaged over the five classes, and the class-wise precision, recall and F1 used for this were also recorded for every model. Parameters are the total model parameters. Inference time is the mean latency of one image (batch size 1) on the GPU through a compiled `tf.function`, after 10 warm-up runs, averaged over the first 200 test images (the host-to-device copy is included).

Model	Position	Accuracy	Precision	Recall	F1	Parameters	Inference Time (ms)
Baseline	–	0.9292	0.9282	0.9274	0.9277	23,597,957	9.38
Channel Attention	P_3	0.9256	0.9262	0.9223	0.9237	23,730,117	9.70
Spatial Attention	P_3	0.9310	**0.9336**	0.9267	0.9295	23,598,056	9.72
Channel+Spatial	P_2	0.9238	0.9222	0.9223	0.9221	23,631,368	9.63
Channel+Spatial	P_3	**0.9328**	0.9326	**0.9296**	**0.9309**	23,730,216	9.81
Channel+Spatial	P_4	0.9256	0.9276	0.9225	0.9246	24,124,520	9.79
Self-Attention	P_3	0.9002	0.8999	0.8972	0.8981	24,909,958	9.89
Additional Attention-1 (ECA)	P_3	0.9238	0.9256	0.9187	0.9214	23,597,962	9.63
Additional Attention-2 (CBAM)	P_3	0.9201	0.9199	0.9155	0.9168	23,730,216	9.80
Additional Attention-3 (Coordinate)	P_3	0.9310	0.9300	0.9285	0.9290	23,698,469	9.68

Channel+Spatial at P_3 comes out on top on three of the four metrics, with spatial attention and Coordinate attention close behind. One test image is worth about 0.0018 in accuracy. Apart from self-attention and the position effect, most differences are therefore only a few images wide, and a second seed would be needed before calling any of them decisive.

22. Graphical Performance Analysis

Appropriate graphs were generated to analyze model behavior.

All six graphical comparisons were generated and are discussed below.

22.1 Training and Validation Accuracy

The following is plotted:

Epoch versus Accuracy

for:

- baseline ResNet50; and
- selected attention models.

Both training and validation accuracy are shown.

The discussion below addresses whether attention:

- accelerates convergence;
- produces smoother learning;
- reduces the train–validation gap;
- leads to overfitting; or
- causes unstable training.

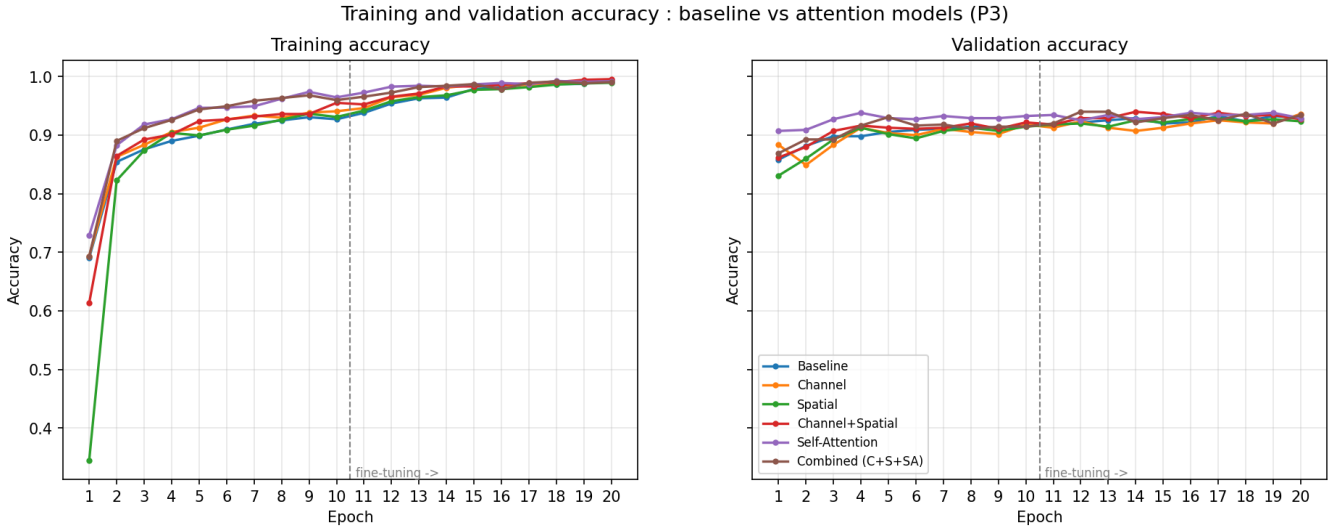


Figure 10: Training and validation accuracy of the baseline and the five attention models of Experiment 1 (dashed line: start of fine-tuning).

Training accuracy climbs to about 0.99 for every model, while validation accuracy flattens at 0.91–0.94 from roughly epoch 4 onward. The last ten epochs mostly widen a train–validation gap of about six points, which is mild overfitting rather than unstable training. Self-attention converges fastest in phase 1 and spatial attention starts slowest (0.345 training accuracy at epoch 1), but by fine-tuning all curves overlap. No attention module clearly reduces the gap.

22.2 Training and Validation Loss

The following is plotted:

$$\text{Epoch} \quad \text{versus} \quad \mathcal{L}_{CE}.$$

The graph includes training and validation loss.
The convergence behavior of the baseline and attention models is compared below.

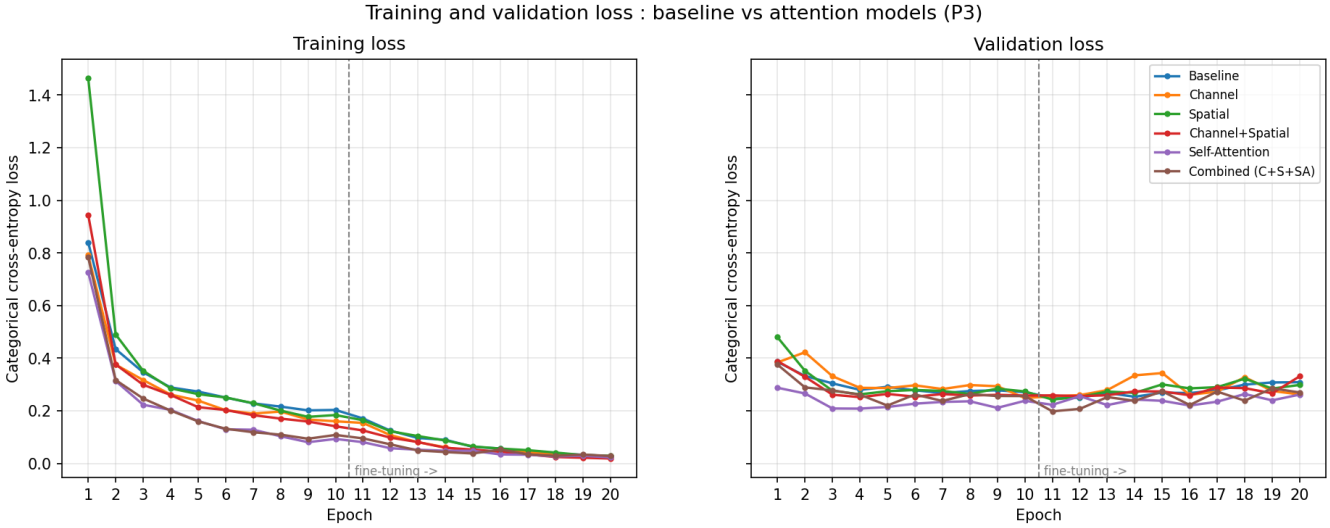


Figure 11: Training and validation categorical cross-entropy for the same six models.

Training loss keeps falling to about 0.02–0.03. Validation loss bottoms out near 0.20–0.26 around the start of fine-tuning and then drifts up (baseline 0.31, A3 0.33 at epoch 20), while validation accuracy stays flat. The models are becoming over-confident on the images they get wrong, an early overfitting sign. This is why the checkpoint with the best validation accuracy, not the last epoch, was used for testing.

22.3 Attention Mechanism versus Accuracy

A bar graph is generated:

$$x = \text{Attention Mechanism}$$

and

$$y = \text{Classification Accuracy}.$$

The entries plotted are:

$$\text{Baseline, Channel, Spatial, Channel+Spatial, Self-Attention, SE, ECA, CBAM.}$$

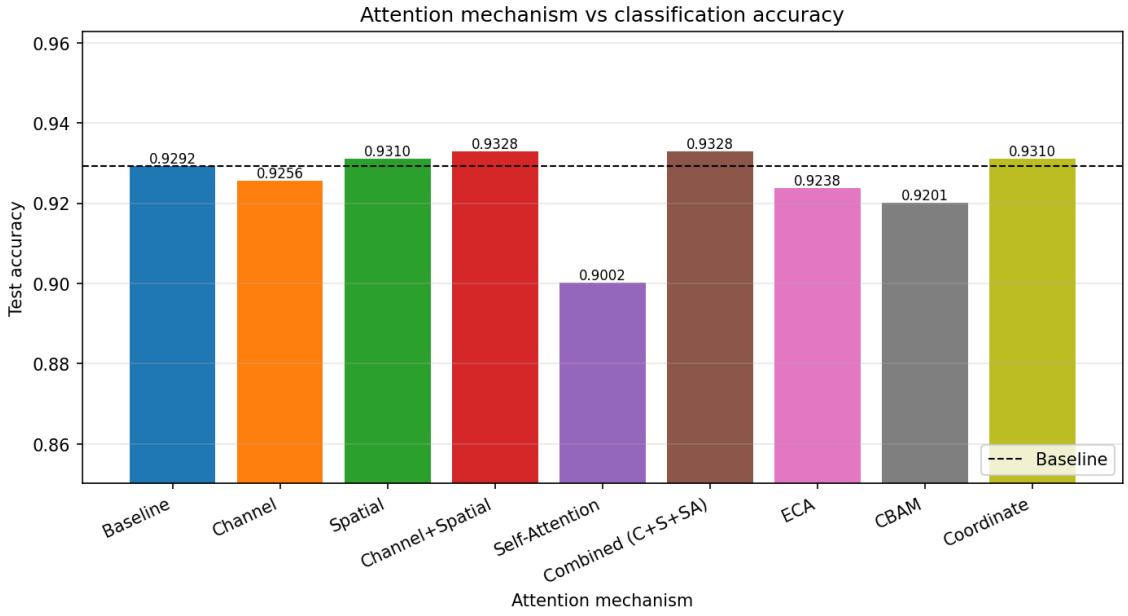


Figure 12: Test accuracy of each attention mechanism at P_3 (dashed line: baseline).

Accuracy sits in a narrow 0.92–0.933 band for everything except self-attention (0.900). Spatial, Channel+Spatial, Combined and Coordinate are at or slightly above the baseline, by one or two test images, while SE, ECA and CBAM are slightly below it. On a strong pretrained backbone, attention is clearly not an automatic improvement.

22.4 Attention Mechanism versus Macro F1

Similarly, the following is plotted:

$$x = \text{Attention Mechanism}$$

and

$$y = \text{Macro F1-score}.$$

This comparison is useful when the class-wise performance is not uniform.

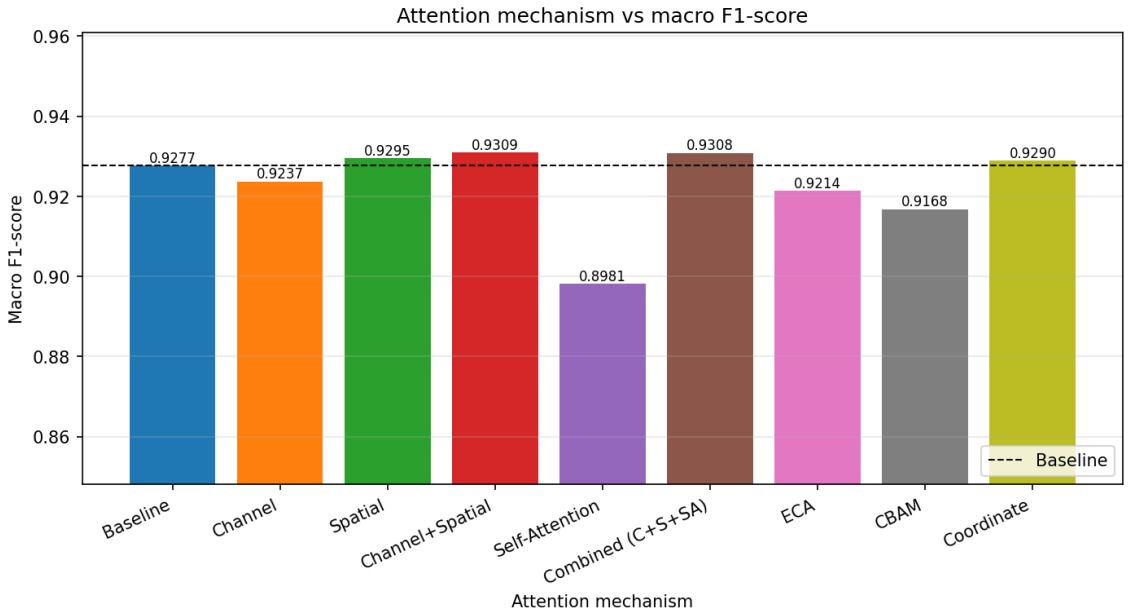


Figure 13: Macro F1-score of each attention mechanism at P_3 (dashed line: baseline).

The F1 ranking is almost the same as the accuracy ranking, and F1 is never more than 0.005 below accuracy. With

only mild class imbalance (95–135 test images per class), the gains and losses are spread over classes rather than hidden in one of them. Section 23 shows that the spread is still not perfectly even.

22.5 Accuracy versus Model Parameters

A scatter plot is created using:

$$x = \text{Total Model Parameters}$$

and

$$y = \text{Test Accuracy}.$$

Each point represents one attention mechanism.
This allows the

$$\boxed{\text{Accuracy–Complexity Trade-off}}$$

to be studied instead of evaluating accuracy independently.

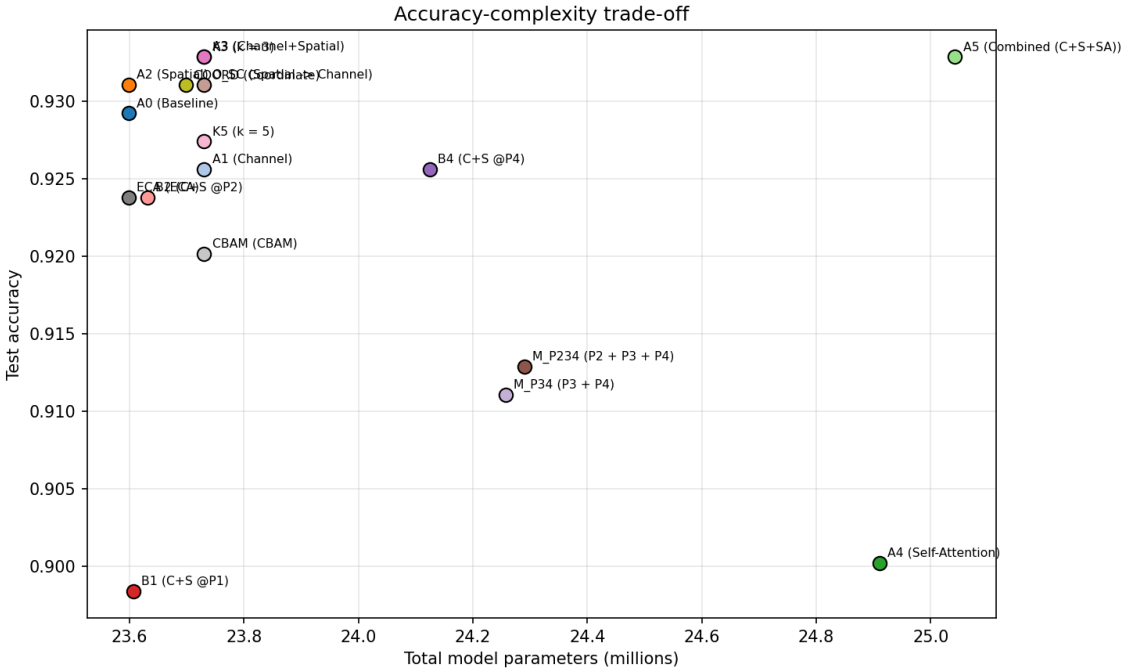


Figure 14: Test accuracy against total parameters for all seventeen trained models.

There is no upward trend. The two largest models (A4 at 24.9M and A5 at 25.0M) give both the worst (0.9002) and the joint-best (0.9328) accuracy, and the 99-parameter spatial module reaches 0.9310. Where the extra parameters go matters far more than how many there are.

22.6 Macro F1 versus Inference Time

The following is plotted:

$$x = \text{Inference Time per Image}$$

and

$$y = \text{Macro F1-score}.$$

The discussion below analyzes whether additional computational cost produces a meaningful increase in classification performance.

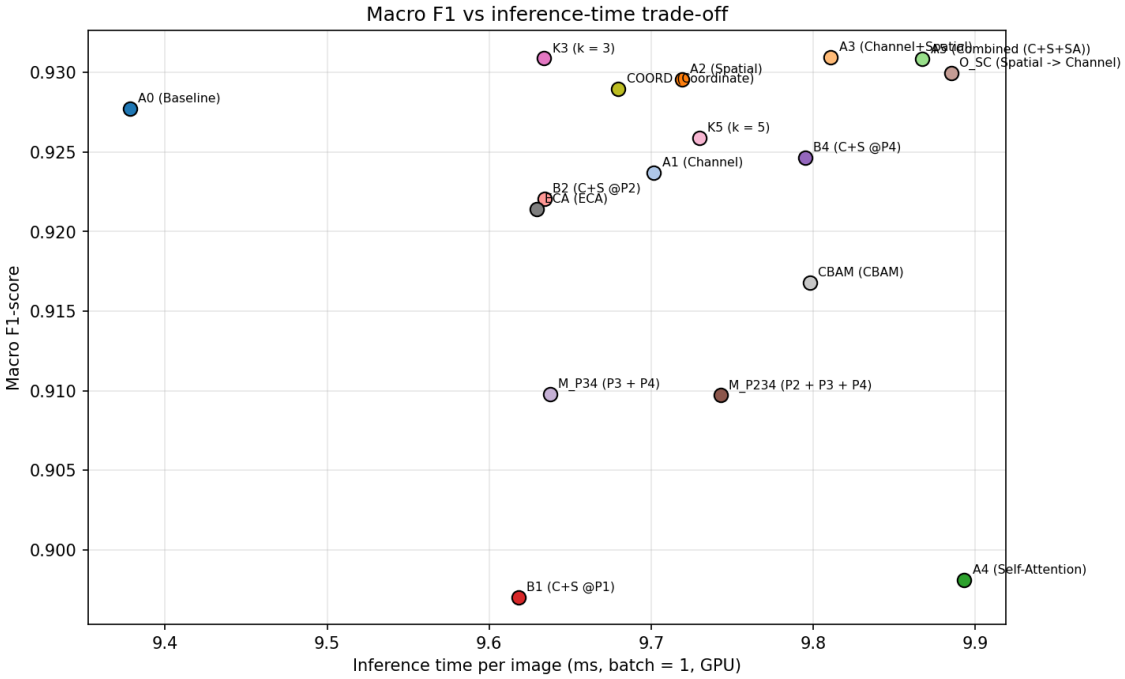


Figure 15: Macro F1 against inference time per image (batch 1, GPU) for all seventeen models.

All models run within 9.38–9.89 ms, so attention adds at most about 5% latency. The best F1 (0.9309) is already reached at 9.63 ms ($k = 3$), while the slowest model (self-attention, 9.89 ms) has the lowest F1. Paying for extra computation did not translate into better F1 in any of these runs.

23. Confusion Matrix Analysis

A confusion matrix was generated for:

1. baseline ResNet50; and
2. the selected attention-based ResNet50.

The confusion matrix contains all five flower classes.

The analysis below covers:

- which classes are most accurately classified;
- which classes are frequently confused;
- which class benefits most from attention;
- whether attention reduces a particular type of confusion; and
- whether the improved overall accuracy is distributed across classes.

The selected attention model is A3 (Channel→Spatial at P_3), the best model of Experiment 1.

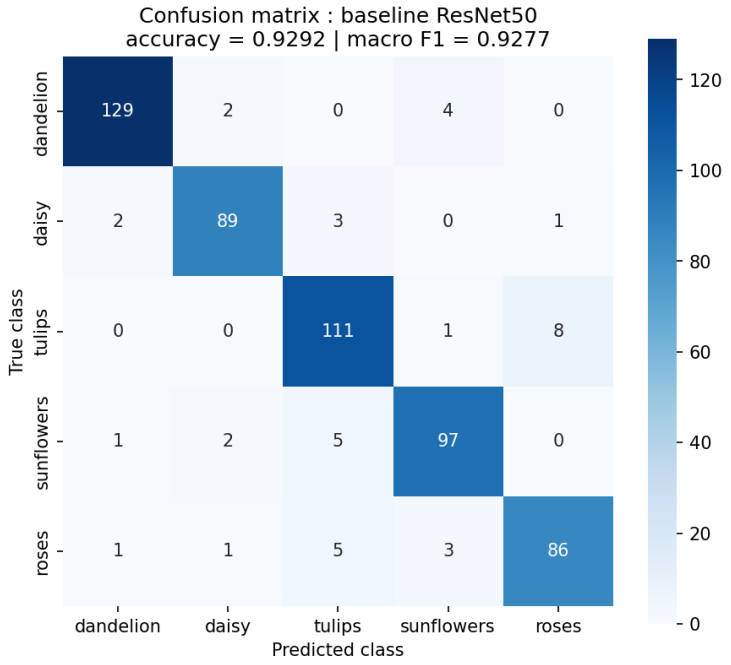


Figure 16: Confusion matrix of the baseline ResNet50 on the test set.

Dandelion is recognised best (129 of 135). The largest single error is tulips predicted as roses (8), followed by roses→tulips (5), sunflowers→tulips (5) and dandelion→sunflowers (4). The tulip–rose mix-up makes sense, since both often appear as cup-shaped red or pink blooms.

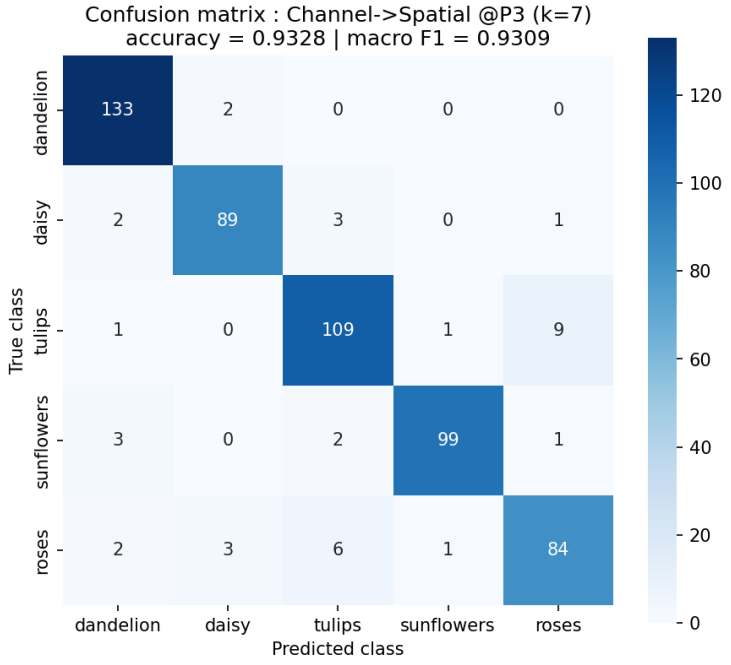


Figure 17: Confusion matrix of the selected attention model A3 (Channel→Spatial at P_3).

A3 fixes the dandelion→sunflower confusion completely (4 to 0) and gets 133 dandelions and 99 sunflowers right. It loses ground on roses (86 to 84 correct) and tulips (111 to 109), with tulips→roses rising to 9. The net gain of two images is the result of some classes improving and others slipping, not a uniform shift.

Class-wise F1	**dandelion**	**daisy**	**tulips**	**sunflowers**	**roses**
Baseline	0.9627	0.9418	0.9098	0.9238	0.9005
A3	0.9638	0.9418	0.9083	0.9612	0.8796
Change	+0.0011	0.0000	−0.0015	**+0.0374**	−0.0209

- **Most accurate class:** dandelion in both models (F1 0.963 / 0.964).

- **Most frequently confused:** tulips and roses, in both directions.
- **Class that benefits most:** sunflowers (+0.037 F1), helped by losing the dandelion→sunflower errors.
- **Confusion reduced:** dandelion→sunflowers. The tulip↔rose confusion did not shrink, it grew slightly.
- **Is the improvement distributed?** No. Roses drop by 0.021 F1, so the small overall gain is concentrated in sunflowers.

24. Qualitative Analysis of Attention

Quantitative classification performance does not directly explain where the network obtains evidence for its prediction.

Qualitative analysis was therefore performed.

The visual analysis includes:

- internal attention maps;
- attention overlays;
- intermediate feature maps; and
- Grad-CAM explanations.

The same input image is used when comparing different models.

For all qualitative figures, the same five representative test images were used: one per class, picked with a fixed seed from the images that both the baseline and A3 classified correctly. Heatmap overlays are min–max normalised per map for visibility, so the raw maps (with colour bars) should be read for absolute values.

The intermediate feature maps of A3 at P_3 are shown below, just before and just after the attention block. Each row shows the mean over all 1024 channels and the four channels with the highest channel-attention weight for that image.

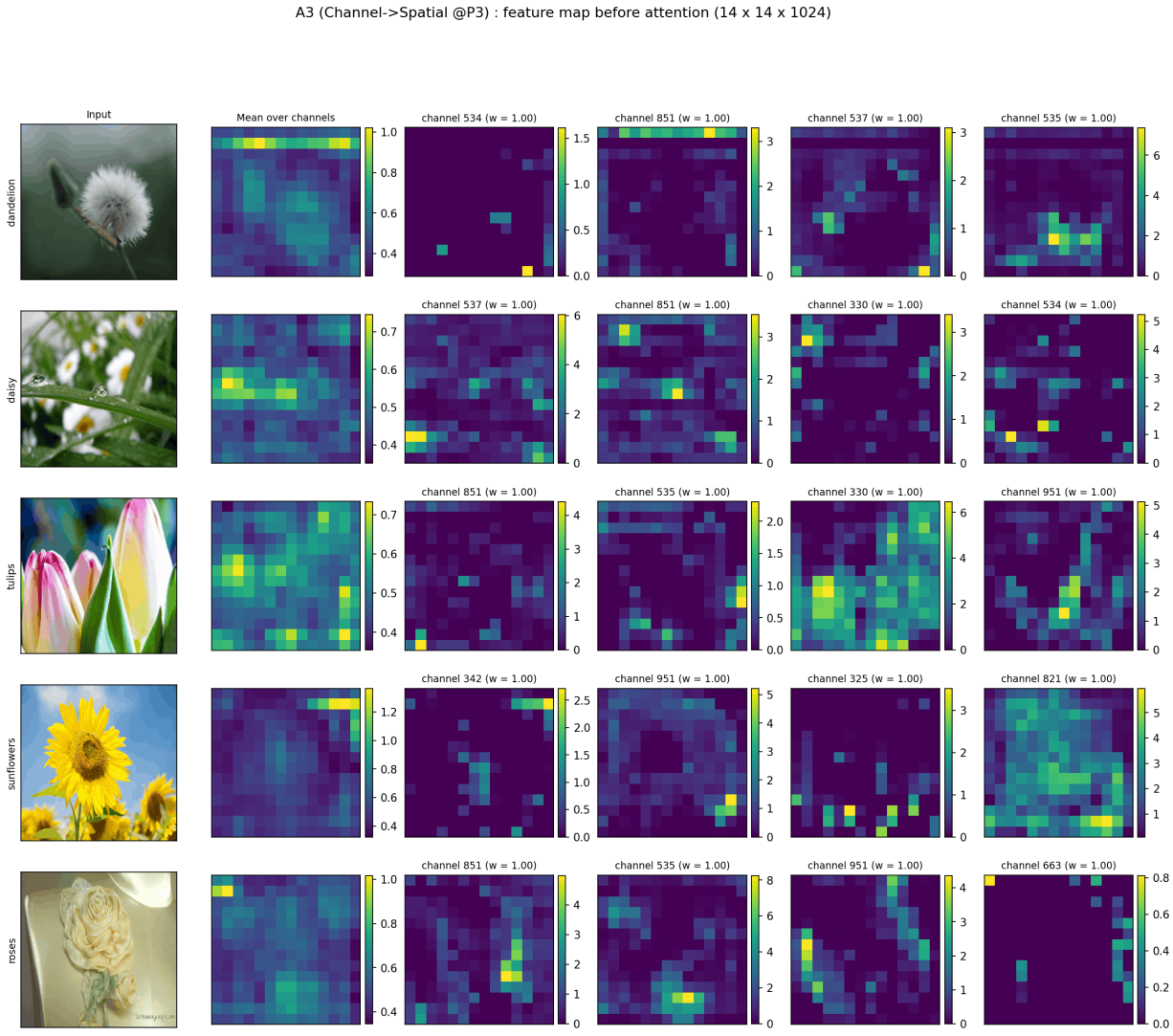


Figure 18: A3: conv4_x feature map before attention (mean over channels and the top-4 weighted channels).

Before attention, the channel-mean map is bright in scattered places that are not always on the flower (the top row in the dandelion image, the top-right corner in the sunflower image). Individual channels are sparse and pick up different parts, for example channel 535 on the lower half of the dandelion head and channel 821 on the sunflower field. The conv4_x representation is already partly object-aware but still mixes in background.

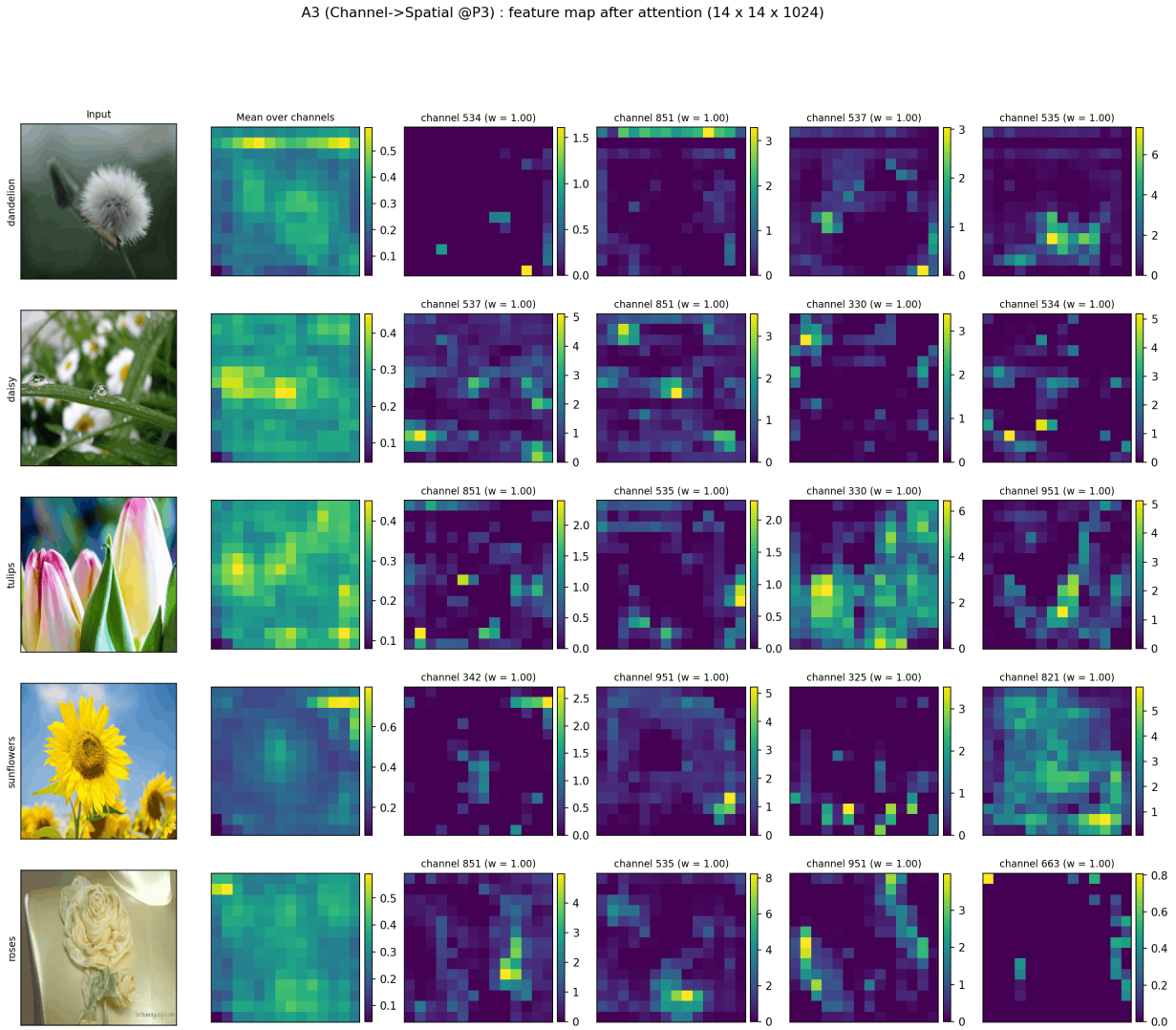


Figure 19: A3: the same feature map after Channel→Spatial attention.

The spatial pattern after attention is almost the same as before, but the channel-mean values shrink by roughly 40–45% (for example, the dandelion peak falls from about 1.0 to 0.55). The top-4 channels look unchanged because their weights are ≈ 1.0. The module mainly dims less useful channels and slightly darkens the borders; it does not redraw where the features are.

25. Grad-CAM Visualization

Gradient-weighted Class Activation Mapping (Grad-CAM) provides a class-specific visualization of the regions contributing to a prediction.

Let

$$A^k$$

represent feature map k from a selected convolutional layer.

For class score y^c, the importance coefficient is:

$$\alpha_k^c = \frac{1}{HW} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}.$$

The Grad-CAM map is:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha^c_k A^k\right).$$

The map is resized to the original image resolution and overlaid on the input.
The following are compared:

Input | Baseline Grad-CAM | Attention Model Grad-CAM

for the same image.
Grad-CAM was computed from the pre-softmax score of the predicted class at `conv5_block3_out`, or at the output of the P_4 attention block for models that have one, so that every model is explained at the same depth.

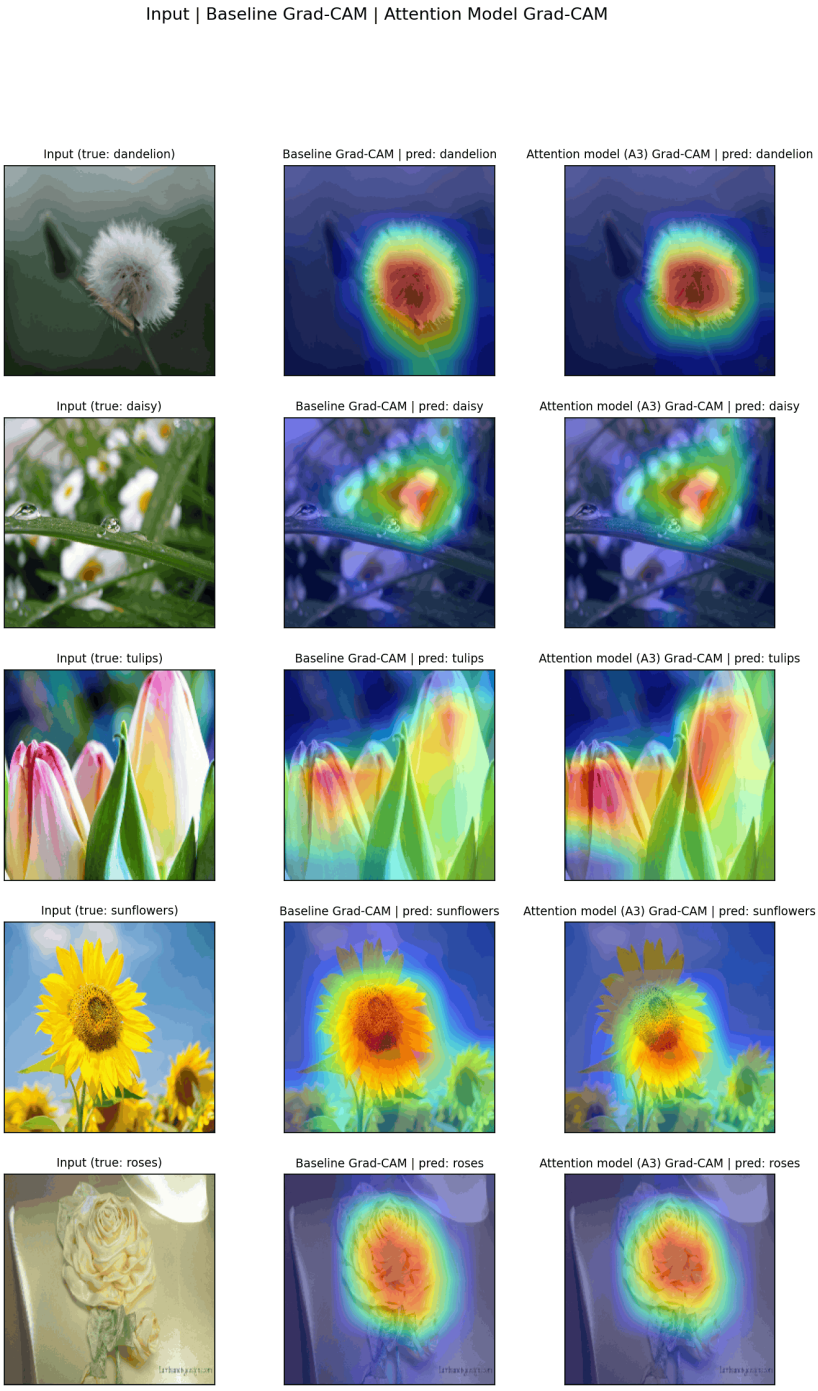


Figure 20: Input — Baseline Grad-CAM — Attention-model (A3) Grad-CAM for the five representative images.

Both models put their strongest activation on the flower in all five images, and for the dandelion, daisy and rose the two maps are nearly identical. Visible differences appear in the tulip image, where A3 spreads more weight onto

the buds on the left, and the sunflower image, where A3 is weaker on the central disc and stronger on the lower petals. Attention at P_3 nudges the evidence but does not change where the network looks, which fits the two-image accuracy difference.

25.1 Grad-CAM Interpretation

The discussion below qualitatively determines whether the attention model:

- focuses more strongly on the flower;
- suppresses irrelevant background regions;
- concentrates on petals;
- concentrates on the central flower structure;
- attends to discriminative edges;
- distributes activation across the entire flower;
- incorrectly focuses on leaves;
- incorrectly focuses on background objects; or
- changes its focus for an incorrect prediction.

The observations below describe visible evidence rather than unsupported claims. For example:

> The baseline network activates both the flower and surrounding leaves, whereas the attention-based model produces stronger activation over the petal region and weaker activation over the background.

Based on the five representative images above and the five extra images in Section 28:

- **Focus on the flower:** both baseline and A3 focus on the flower in all correctly classified images. A3 is not visibly more flower-focused overall.
- **Background suppression:** in the crowded tulip-bed scene (Section 28, row 3), A3 is tighter on the tulips while the baseline spreads to the right-hand background. The other images show no clear difference.
- **Petals and flower centre:** for the dandelion and rose, both models cover the whole head. For the sunflower, the baseline sits on the central disc while A3 shifts towards the petals below it.
- **Leaves and background errors:** on the misclassified daisy, A3 activates the out-of-focus bright spots in the background and predicts tulips, while the baseline stays on the flower centre and predicts correctly.
- **Change of focus on errors:** on the misclassified images, A3's map breaks into several small blobs instead of one compact region.

26. Internal Attention Maps

An important objective of this experiment is to distinguish between:

Internal Attention

and

Post-hoc Explanation

because they are not the same.

An attention module directly computes weights during the forward pass.

Grad-CAM, in contrast, is generated after prediction using gradients.

26.1 Channel-Attention Visualization

For channel attention:

$$M_c(F) = [m_1, m_2, \ldots, m_C].$$

The learned channel importance values were visualized using a bar graph.

Instead of displaying every channel when C is large, the top-K channels are shown.

The value used was:

$$K = 20.$$

The discussion below comments on whether:

- a small number of channels receive strong weights;

- many channels contribute similarly; or
- channel weighting changes substantially for different input images.

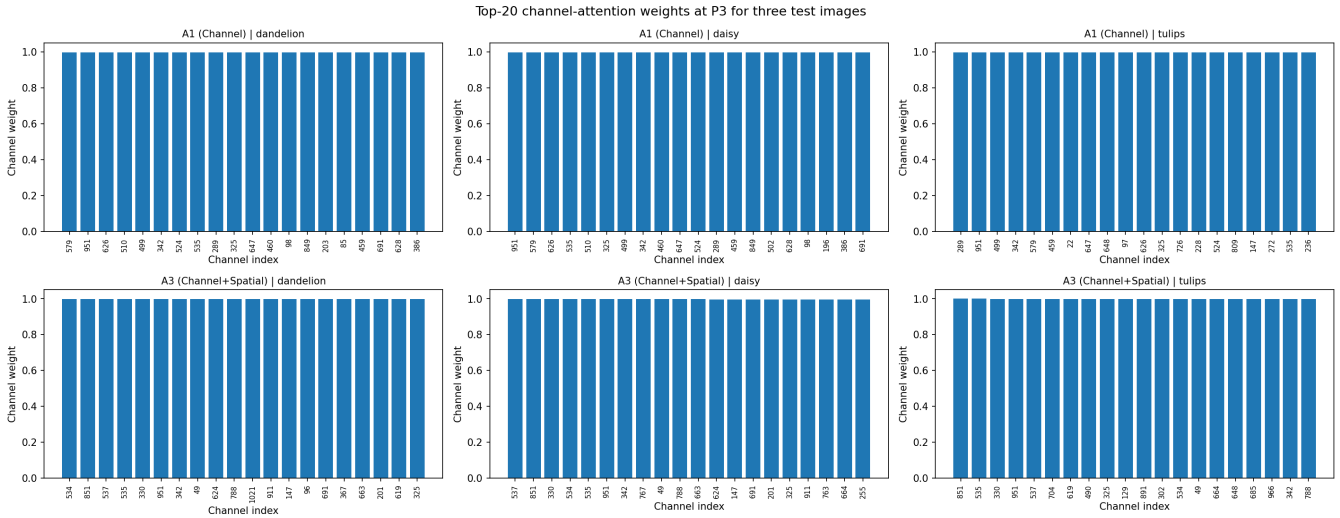


Figure 21: Top-20 channel-attention weights at P_3 for three test images (top: A1, bottom: A3).

In both models the top-20 weights are all saturated at about 1.0, so for these channels the gate is essentially fully open. Several indices come back across images (951, 579, 342 and 626 in A1; 851, 537, 330 and 535 in A3), while the tulip image brings in a partly different set. Many channels contribute equally at the top, and the weighting changes only moderately between images. Together with the drop in mean activation in Section 24, this means the module works by turning other channels down rather than boosting a few.

26.2 Spatial-Attention Visualization

For spatial attention:

$$M_s(F) \in \mathbb{R}^{H \times W}.$$

The attention map was resized to:

$$224 \times 224$$

and overlaid on the input image.
The following is displayed:

Input | Spatial Attention Map | Attention Overlay

The image regions receiving high attention values are identified below.

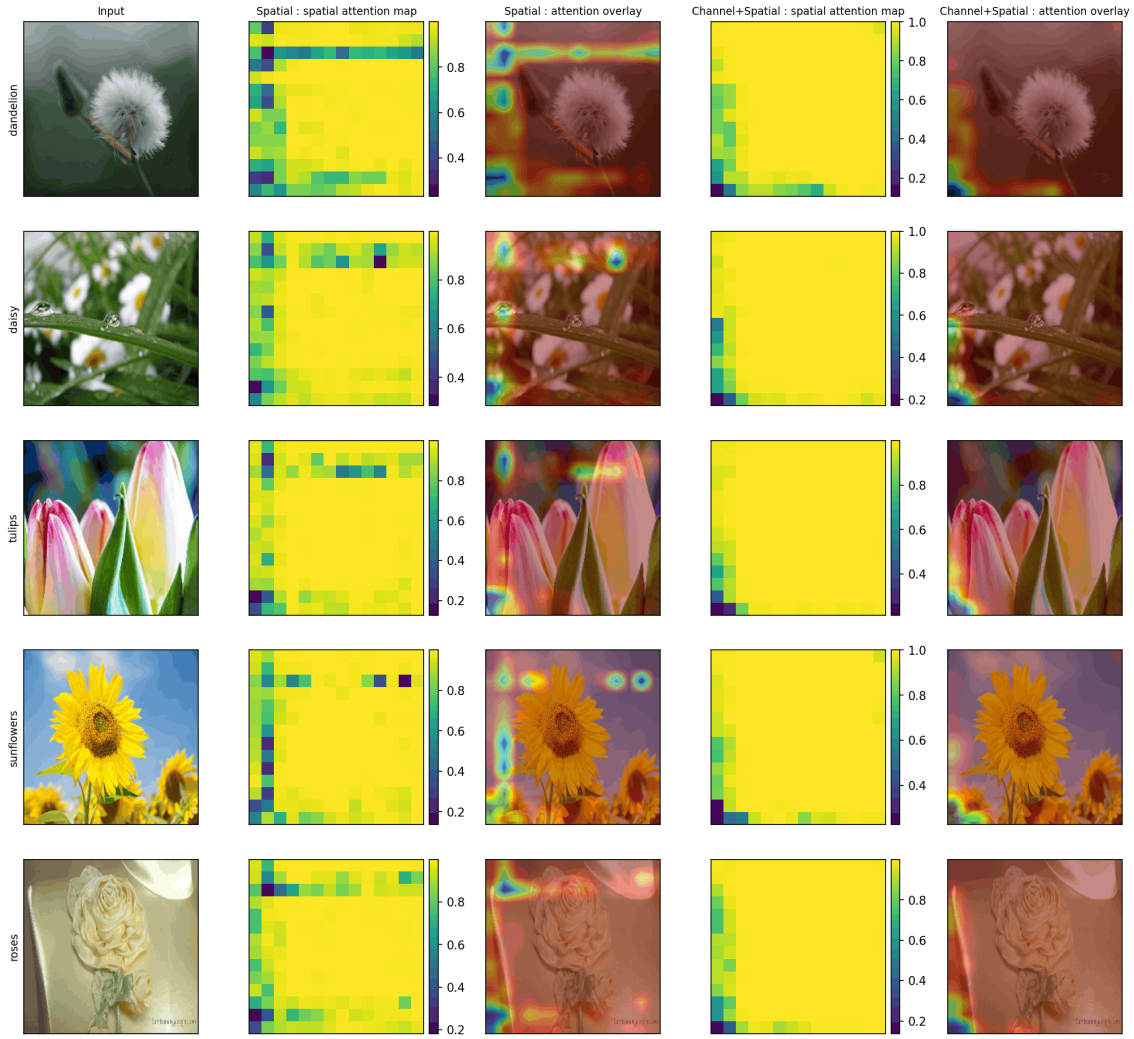


Figure 22: Input — Spatial Attention Map — Attention Overlay for spatial attention (A2) and Channel+Spatial attention (A3) at P_3.

The maps stay close to 1.0 over almost the whole 14×14 grid. The low values sit on the borders: the left column, top and bottom rows and the bottom-left corner for A2, and only the left and bottom edges for A3. They are not shaped like the flowers, which points to the 7×7 convolution reacting to the zero padding at the image edge. The overlays exaggerate this because of the min–max scaling. In effect, the gate learned a near-identity with border damping, which is consistent with its small effect on accuracy.

26.3 Self-Attention Visualization

For self-attention, the following was visualized:

- attention associated with a selected query location;
- mean attention across query positions; and
- maximum attention across query positions.

The visualization was reshaped into a spatial form and, when appropriate, resized to the image resolution.

The analysis below covers how distant regions interact within the feature map.

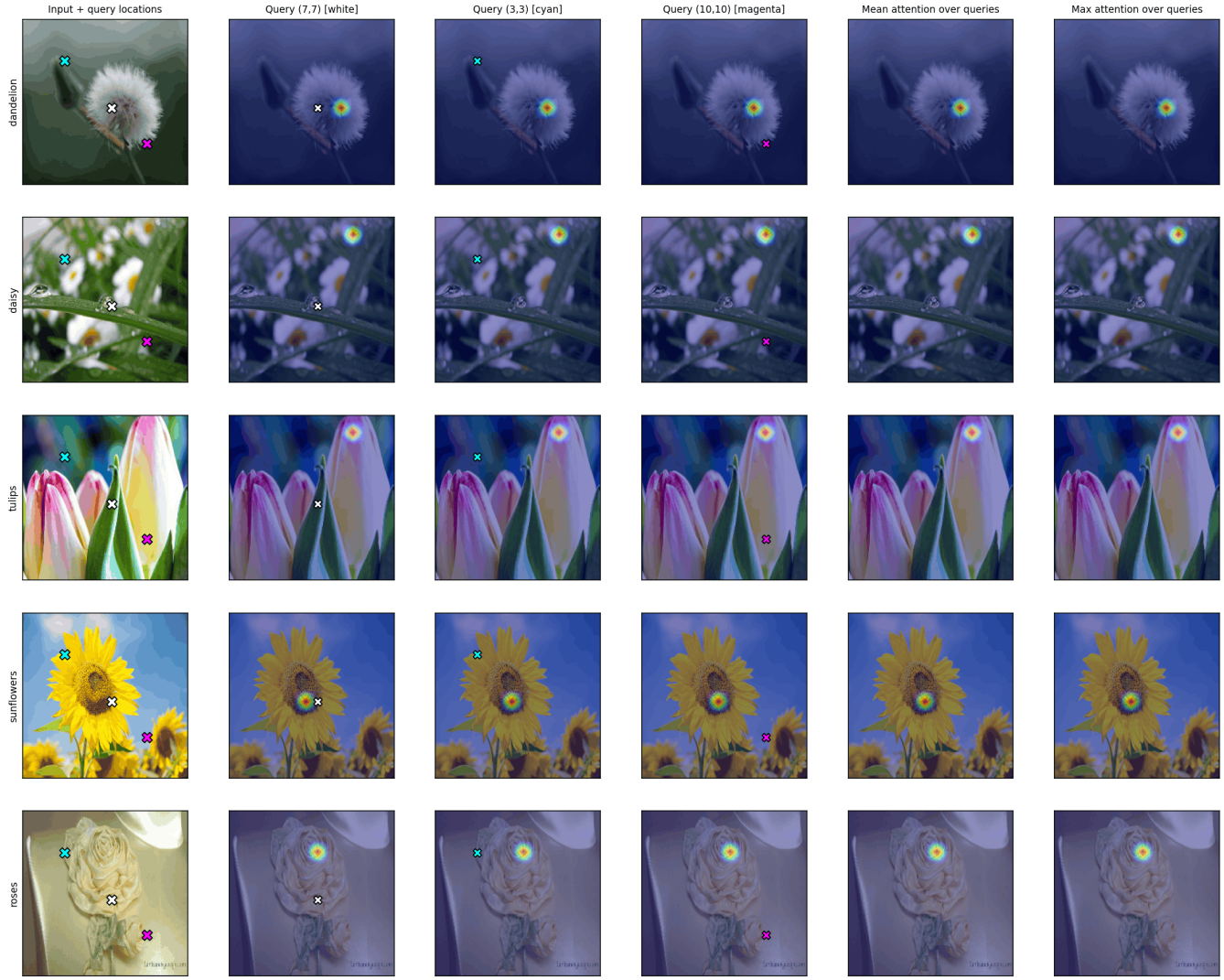


Figure 23: A4 self-attention at P_3: response for three query locations, and the mean and maximum over all queries ($\gamma = 0.065$).

Whatever the query location (centre, top-left or bottom-right), each image produces essentially the same map: one hot key position, usually on the flower itself (the dandelion head, sunflower disc, rose centre or a tulip tip). The mean and max maps match the single-query maps, so the attention has collapsed to a query-independent pattern. Distant regions therefore interact only by all reading from that one location, and with a learned γ of 0.065 this global signal is a small correction on top of the residual path.

27. Attention Position Visualization

The same test image is used to visualize spatial attention at different ResNet50 stages.
The following are generated:

$$M_s^{P_2}, \qquad M_s^{P_3}, \qquad M_s^{P_4}.$$

The maps are displayed as:

$$\boxed{\text{Input} \mid M_s^{P_2} \mid M_s^{P_3} \mid M_s^{P_4}}$$

The objective is to visually study how attention changes as feature representations become deeper.
Observations include:

- P_2 responding strongly to edges, colors, textures, and fine structures;

- P_3 responding to flower parts and intermediate object structures;
- P_4 responding to larger semantic flower regions.

These observations are based on the experimental visualizations below.

The maps come from three separately trained models, B2, B3 (= A3) and B4, each with Channel→Spatial attention at one position.

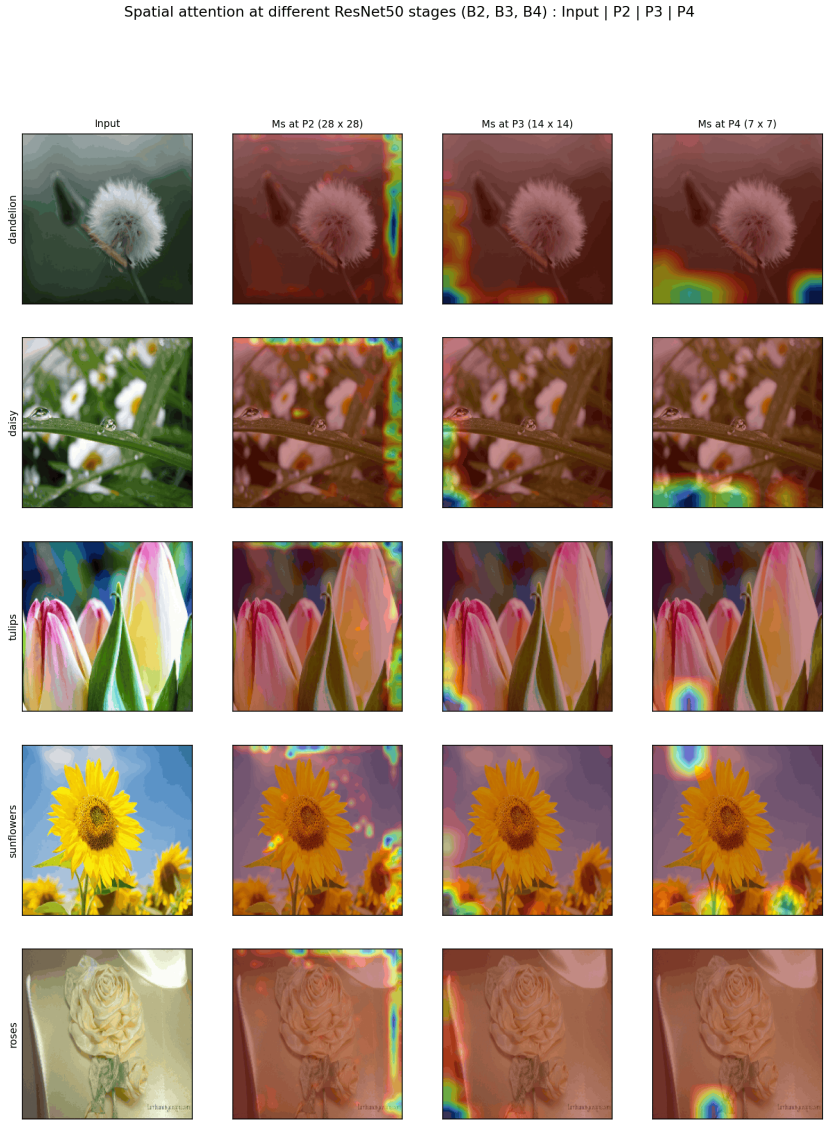


Figure 24: Spatial-attention overlays at P_2 (28×28), P_3 (14×14) and P_4 (7×7) for the same images.

At P_2 the low values form thin streaks along the top and right edges and some fine texture, at P_3 they run along the left and bottom borders, and at P_4 they become coarse blobs, mostly over the lower part of the image (stems, leaves and the lower background in the daisy, tulip and rose images). The maps get more region-like as depth increases, as expected, but at every depth they act as suppressors of edges or regions rather than highlighting the flower itself.

28. Qualitative Comparison Across Attention Mechanisms

At least:

$$\boxed{\text{3 correctly classified images}}$$

and

$$\boxed{\text{2 incorrectly classified images}}$$

were selected.

For every selected image, the behavior of multiple models is compared.

The arrangement used is:

Input Image
↓
Baseline Grad-CAM
Channel-Attention Grad-CAM
Spatial-Attention Map
Channel+Spatial Grad-CAM
Self-Attention Response

The images are arranged horizontally in the report.
Three images that both the baseline and A3 classified correctly and two images that A3 misclassified were chosen with a fixed seed. They are arranged horizontally, one row per image.

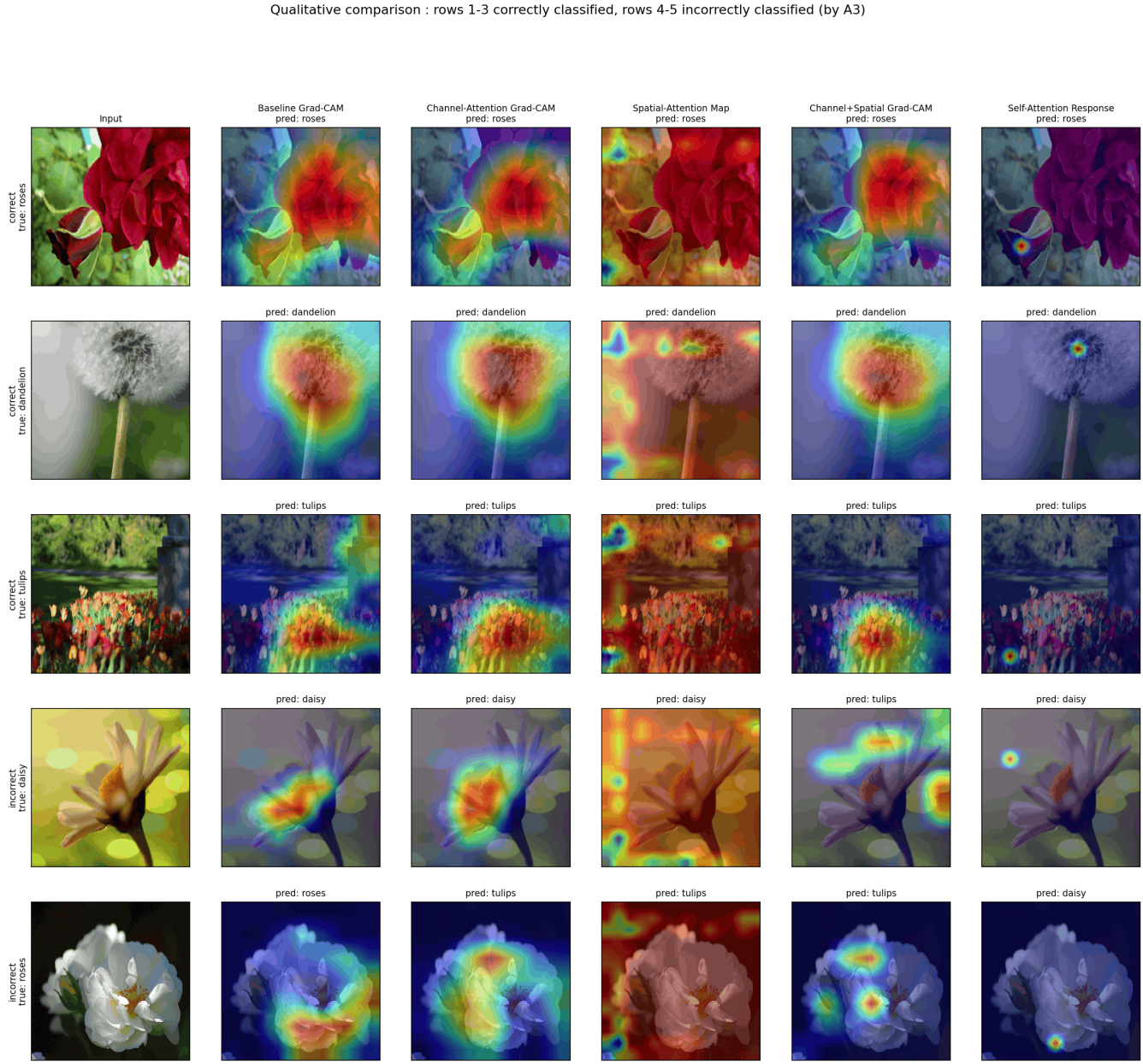


Figure 25: Rows 1–3 correctly classified, rows 4–5 misclassified by A3: Input, Baseline / Channel / Channel+Spatial Grad-CAM, Spatial-attention map and Self-attention response.

On the three correct images all Grad-CAMs cover the flower, with A3 tighter than the baseline on the tulip bed. On the daisy (row 4), the baseline and channel model focus on the flower centre and are right, while A3 fires on the bright background spots and predicts tulips. On the white rose (row 5), the baseline looks at the lower petals and

is right, while the channel and Channel+Spatial models break into fragments and predict tulips. The spatial map shows the same border pattern in every row, so it does not explain the errors; the Grad-CAM fragmentation does.

28.1 Qualitative Criteria

The discussion below covers:

- foreground localization;
- background suppression;
- localization of petals;
- localization of the flower center;
- activation compactness;
- activation coverage;
- incorrectly attended regions;
- differences between correctly and incorrectly classified samples;
- differences between shallow and deep attention; and
- consistency of attention across different images.
- **Foreground localisation:** every model localises the flower on all correctly classified images. The differences are in spread, not in location.
- **Background suppression:** visible only in the tulip-bed scene for A3. The internal spatial maps damp borders, not background as such.
- **Petals and centre:** channel attention and the baseline usually centre on the disc or head. A3 sometimes moves to the petals (the sunflower).
- **Compactness and coverage:** correct predictions give one compact blob, while wrong ones give several smaller, scattered blobs.
- **Incorrectly attended regions:** out-of-focus background highlights (the daisy) and petal edges (the white rose).
- **Correct vs incorrect:** the difference shows up in Grad-CAM, not in the internal spatial maps, which look alike in both cases.
- **Shallow vs deep:** P_2 maps are fine-grained and edge-like, while P_4 maps are coarse and region-like (Section 27).
- **Consistency:** the internal maps are very consistent across images (always border-shaped, and a single key for self-attention), which is itself a sign that they carry little image-specific information.

29. Recommended Visualization Layout

For one representative test image, a figure was generated in the following format:

Input	Baseline Grad-CAM	Channel Grad-CAM	Spatial Attention Map	Channel+Spatial Grad-CAM

For the additional attention comparison, the figure was extended to:

Input|Baseline|SE|ECA|CBAM|Self-Attention

using appropriate visualization methods.

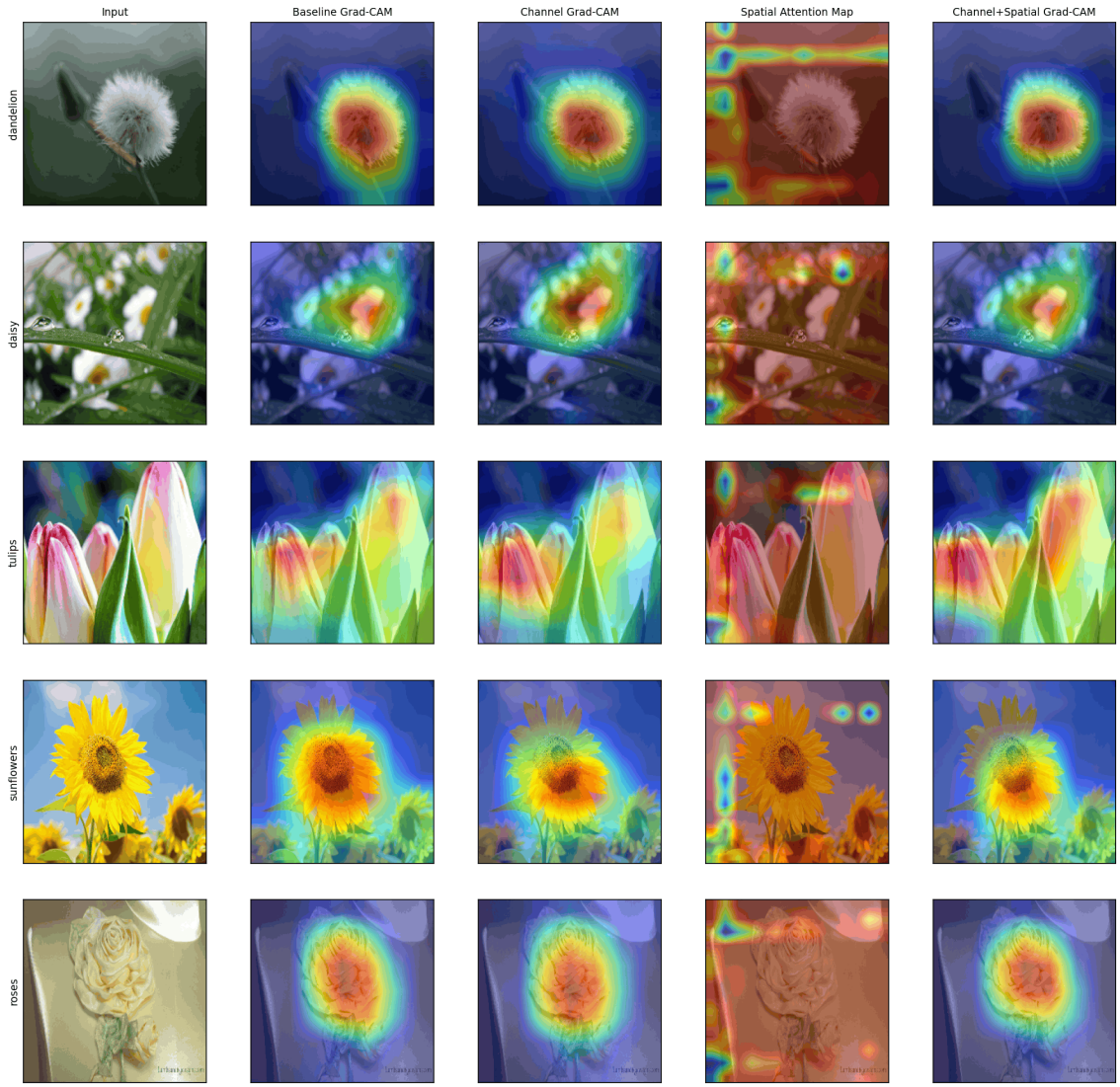


Figure 26: Input — Baseline Grad-CAM — Channel Grad-CAM — Spatial Attention Map — Channel+Spatial Grad-CAM for the five representative images.

Channel-attention Grad-CAM is almost a copy of the baseline on all five images, and Channel+Spatial differs only on the tulip and sunflower. The spatial attention map again shows the border pattern instead of the flower. Read together, the internal gate is near-uniform inside the image, so the post-hoc explanation barely changes.

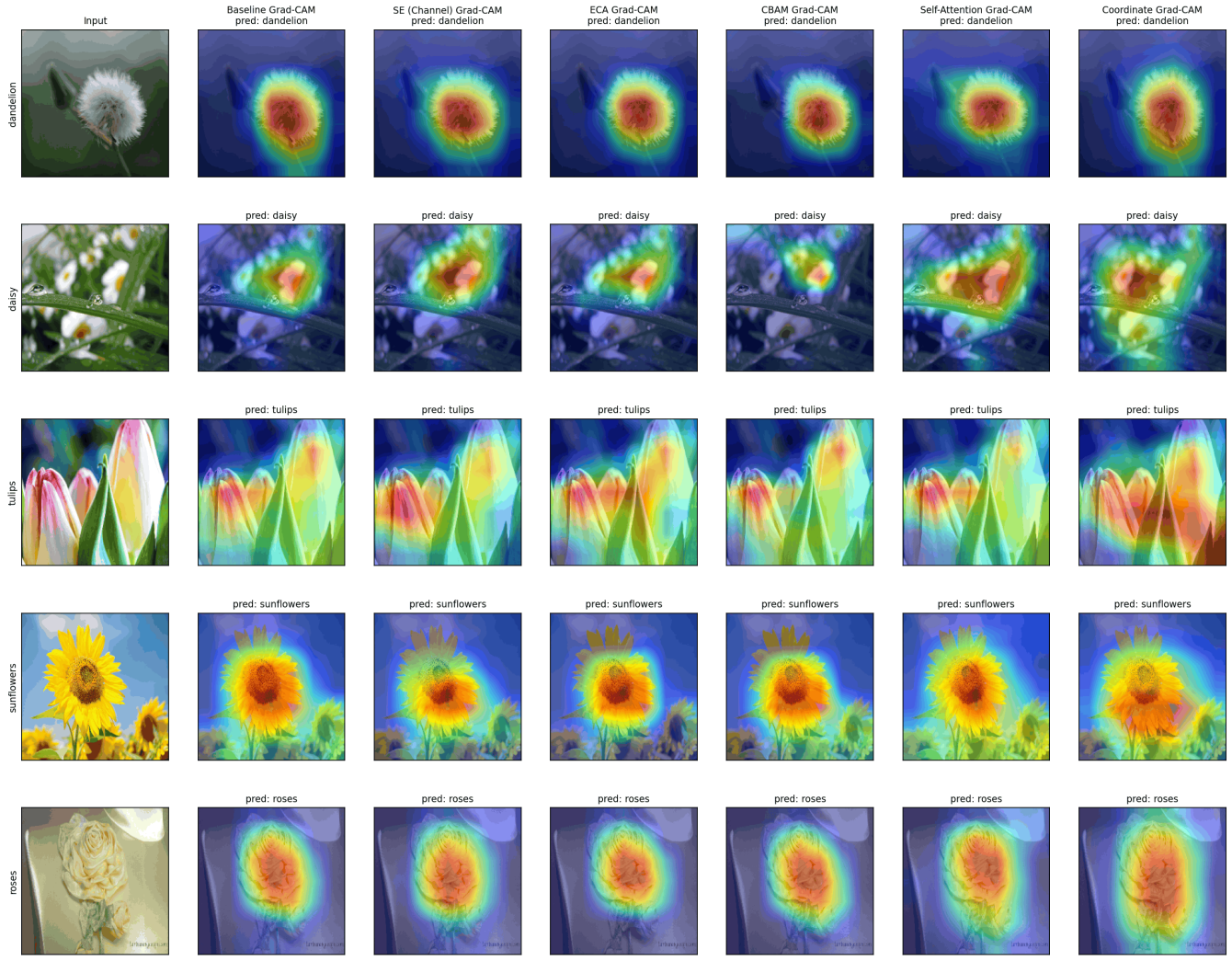


Figure 27: Extended layout (Grad-CAM): Input — Baseline — SE — ECA — CBAM — Self-Attention — Coordinate Attention.

All six models classify these five images correctly with similar maps, so the differences are subtle. The daisy image separates them most: CBAM gives the most compact map on a single flower, self-attention the most spread-out one (along the leaf and several flowers), and Coordinate attention extends downwards. ECA looks almost identical to SE, which matches their near-equal accuracy despite the 26,000× difference in parameters.

30. Experimental Analysis

The final report contains three forms of analysis.

30.1 Quantitative Analysis

The following are reported:

$$\boxed{\text{Accuracy} + \text{Precision} + \text{Recall} + \text{Macro F1}}$$

The observations recorded for every trained model are listed below. Training accuracy and loss are from the last epoch, on augmented data. Test values use the best-validation checkpoint.

Key	Model	Train Acc	Val Acc (final)	Val Acc (best)	Test Acc	Train Loss	Val Loss	Precision	Recall	Macro F1
A0	Baseline	0.9938	0.9291	0.9327	0.9292	0.0276	0.3087	0.9282	0.9274	0.9277
A1	Channel (SE) @P_3	0.9926	0.9364	0.9364	0.9256	0.0255	0.2623	0.9262	0.9223	0.9237
A2	Spatial @P_3	0.9895	0.9236	0.9291	0.9310	0.0290	0.2983	0.9336	0.9267	0.9295
A3	C→S @P_3, $k = 7$	0.9957	0.9273	0.9400	0.9328	0.0188	0.3311	0.9326	0.9296	0.9309
A4	Self-Attention @P_3	0.9922	0.9291	0.9382	0.9002	0.0228	0.2617	0.8999	0.8972	0.8981
A5	C→S→SA @P_3	0.9907	0.9345	0.9400	0.9328	0.0284	0.2698	0.9345	0.9282	0.9308
B1	C→S @P_1	0.9895	0.9218	0.9236	0.8984	0.0330	0.3080	0.8971	0.8983	0.8970
B2	C→S @P_2	0.9914	0.9182	0.9255	0.9238	0.0301	0.3027	0.9222	0.9223	0.9221
B4	C→S @P_4	0.9953	0.9273	0.9273	0.9256	0.0113	0.3554	0.9276	0.9225	0.9246
M_P34	C→S @$P_3 + P_4$	0.9969	0.9036	0.9345	0.9111	0.0100	0.5361	0.9129	0.9088	0.9098
M_P234	C→S @$P_2 + P_3 + P_4$	0.9934	0.9164	0.9236	0.9129	0.0176	0.3624	0.9088	0.9116	0.9097
O_SC	S→C @P_3	0.9891	0.9036	0.9255	0.9310	0.0350	0.4183	0.9304	0.9296	0.9300
K3	C→S @P_3, $k = 3$	0.9910	0.9164	0.9327	0.9328	0.0272	0.4193	0.9326	0.9296	0.9309
K5	C→S @P_3, $k = 5$	0.9922	0.9273	0.9273	0.9274	0.0255	0.3451	0.9261	0.9269	0.9259
ECA	ECA @P_3	0.9942	0.9127	0.9218	0.9238	0.0196	0.3567	0.9256	0.9187	0.9214
CBAM	CBAM @P_3	0.9875	0.9236	0.9236	0.9201	0.0360	0.3937	0.9199	0.9155	0.9168
COORD	Coordinate @P_3	0.9848	0.9255	0.9327	0.9310	0.0412	0.4483	0.9300	0.9285	0.9290

Every model fits the training set almost perfectly (0.985–0.997), while test accuracy stays between 0.898 and 0.933. The best validation accuracy is often higher than the test accuracy (A4: 0.9382 vs 0.9002), a reminder that choosing a checkpoint on 550 validation images is itself a noisy step.

30.2 Complexity Analysis

The following are reported:

$$\boxed{\text{Parameters} + \text{Inference Time}}$$

Model	Trainable (phase 1)	Trainable (phase 2)	Total Parameters	Attention Params	Inference (ms/img)
A0 Baseline	10,245	14,963,717	23,597,957	0	9.38
A1 Channel (SE)	142,405	15,095,877	23,730,117	132,160	9.70
A2 Spatial	10,344	14,963,816	23,598,056	99	9.72
A3 C→S @P_3	142,504	15,095,976	23,730,216	132,259	9.81
A4 Self-Attention	1,322,246	16,275,718	24,909,958	1,312,001	9.89
A5 Combined	1,454,505	16,407,977	25,042,217	1,444,260	9.87
B1 C→S @P_1	18,808	14,972,280	23,606,520	8,563	9.62
B2 C→S @P_2	43,656	14,997,128	23,631,368	33,411	9.63
B4 C→S @P_4	536,808	15,490,280	24,124,520	526,563	9.79
M_P34	669,067	15,622,539	24,256,779	658,822	9.64
M_P234	702,478	15,655,950	24,290,190	692,233	9.74
O_SC S→C	142,504	15,095,976	23,730,216	132,259	9.89
K3 ($k = 3$)	142,424	15,095,896	23,730,136	132,179	9.63
K5 ($k = 5$)	142,456	15,095,928	23,730,168	132,211	9.73
ECA	10,250	14,963,722	23,597,962	5	9.63
CBAM	142,504	15,095,976	23,730,216	132,259	9.80
Coordinate	110,693	15,064,165	23,698,469	100,512	9.68

Attention adds between 5 and 1.44M parameters, which is at most about 6% of ResNet50, and at most 0.51 ms per image. GFLOPs stayed at about 7.75 for every measured model except self-attention (8.35), the only module whose cost grows with N^2. Computationally, attention at P_3 is cheap; the open question is only whether it helps.

30.3 Qualitative Analysis

The following are reported:

$$\boxed{\text{Grad-CAM} + \text{Attention Maps} + \text{Visual Interpretation}}$$

The overall evaluation is therefore:

$$\boxed{\text{Quantitative} + \text{Computational} + \text{Qualitative}}$$

All required visualisations are included above: sample images (Section 2), training curves and performance graphs (Section 22), confusion matrices (Section 23), feature maps before and after attention (Section 24), Grad-CAM (Sections 25, 28 and 29), channel weights, spatial maps and the self-attention response (Section 26), and maps at

P_2, P_3 and P_4 (Section 27). The qualitative picture agrees with the numbers: the learned internal maps are close to uniform inside the image, the Grad-CAM maps change only slightly, and the accuracy changes are correspondingly small.

31. Experimental Protocol

To ensure fair comparison, all ablation experiments used:

- identical training images;
- identical validation images;
- identical testing images;
- identical random seed where possible;
- identical image preprocessing;
- identical augmentation;
- identical backbone;
- identical classifier;
- identical optimizer;
- identical batch size;
- identical number of epochs;
- identical learning-rate schedule;
- identical fine-tuning procedure; and
- identical evaluation metrics.

Only the experimental factor under investigation was changed.

For example, when studying attention position:

Only Attention Position Changed

and when studying attention type:

Only Attention Type Changed.

This protocol was followed for every run. The same saved split and the same preprocessing were used throughout, and the augmentation layers and random seed (42) were re-created before each model. The backbone, classifier, Adam optimiser, batch size, two-phase 10+10 epoch schedule, learning rates and evaluation code were shared by all seventeen trained models. The only sources of variation left were the factor under study and the non-deterministic GPU kernels, which TensorFlow does not make deterministic by default.

32. Minimum Mandatory Ablation Set

The following experiments were completed as the minimum mandatory set:

1. Baseline pretrained ResNet50.
2. Channel attention at P_3.
3. Spatial attention at P_3.
4. Channel+Spatial attention at P_3.
5. Channel+Spatial attention at P_2.
6. Channel+Spatial attention at P_4.
7. Self-attention at P_3.
8. Grad-CAM comparison between baseline ResNet50 and one selected attention model.
9. Spatial-attention visualization at P_2, P_3, and P_4.
10. Graphical comparison of the classification performance of the attention models.

This minimum set allowed

Attention Type

and

Attention Position

to be studied.

All ten items were completed: models A0, A1, A2, A3, B2, B4 and A4 (items 1–7); the baseline vs A3 Grad-CAM comparison (Section 25); spatial maps at P_2, P_3 and P_4 (Section 27); and the graphical comparison (Section 22).

Beyond the minimum set, P_1 (B1), multiple positions, attention order, kernel size and four extra mechanisms were also run.

33. Additional Exploration

For extended experimentation, at least two mechanisms were selected from:

SE, ECA, CBAM, Coordinate Attention, Self-Attention

and integrated with the same ResNet50 backbone.

The same

- dataset split;
- preprocessing;
- augmentation;
- optimizer;
- epochs;
- fine-tuning strategy; and
- attention position

were maintained.

The additional investigation includes both:

Quantitative Comparison

and

Qualitative Comparison.

SE, ECA, CBAM, Coordinate Attention and self-attention were all integrated at P_3 under the identical protocol. The quantitative comparison is in Section 18, and the qualitative one is in the extended Grad-CAM layout of Section 29.

34. Source Code

The repository contains a single Kaggle notebook implementing the complete pipeline described in this report, covering data loading and the fixed split, the rebuilt ResNet50 with insertable attention modules (channel, spatial, channel–spatial, self-attention, ECA, CBAM and Coordinate Attention), two-phase training of all seventeen models, evaluation, Grad-CAM, the internal attention-map visualisations, and every table and figure used here. It is available at:

https://github.com/Ashwin271206/dl-lab/tree/main/Lab%208%20&%209

The repository contains a single Kaggle notebook covering all tasks experimented in this report, along with the generated figures and outputs.

35. Discussion

1. **What is the purpose of residual connections in ResNet50?**
   They let each block learn a correction on top of an identity path, so gradients reach the early layers and a 50-layer network still trains. The same idea made it safe to insert attention here: the self-attention module starts at $\gamma = 0$, so the pretrained path is untouched at first, and it only grew to 0.065 during training.
2. **What does conv2_x represent compared with conv5_x?**
   conv2_x ($56 \times 56 \times 256$) still encodes local edges, colours and textures at almost full detail. conv5_x ($7 \times 7 \times 2048$) encodes coarse, class-level parts of the object. The maps in Section 27 show the same contrast: thin edge streaks at P_2, large blobs at P_4.
3. **Why does the spatial resolution decrease with network depth?**
   Each stage begins with a stride-2 convolution, which halves the height and width. This lets every later filter see a larger part of the image and keeps the computation manageable as the number of channels grows.

4. **Why does the number of feature channels increase with network depth?**
   As spatial detail is pooled away, the network needs more channels to describe the larger number of possible part combinations at a coarse scale. Doubling the channels while halving each spatial side also keeps the cost per stage roughly balanced.
5. **What is meant by low-level, intermediate, and high-level features?**
   Low-level features respond to edges, colour and texture; intermediate features to repeated patterns and flower parts such as petal arrangements; and high-level features to whole-object, class-specific evidence. They roughly correspond to P_1/P_2, P_3 and P_4 respectively.
6. **What is the purpose of an attention mechanism?**
   To re-weight features depending on the input, so that useful channels or locations are kept and less useful ones are damped. On a pretrained backbone it acts as a small learnable adaptor rather than a new feature extractor.
7. **What does channel attention attempt to learn?**
   Which feature detectors matter for the current image. In our models it mostly learned to keep a fairly stable set of channels fully open (weights ≈ 1.0, with indices like 951 and 342 recurring) and to turn the rest down, which cut the mean activation by roughly 40%.
8. **What does spatial attention attempt to learn?**
   Where in the feature map the useful evidence lies. Here it learned an almost uniform map with damped borders rather than a flower-shaped mask, which suggests the conv4_x features were already good enough that little spatial re-weighting was needed.
9. **How is channel attention different from spatial attention?**
   Channel attention gives one weight per channel, shared over all locations (C values, 132,160 parameters at P_3). Spatial attention gives one weight per location, shared over all channels ($H \times W$ values, only 99 parameters). The first chooses *what*, the second chooses *where*.
10. **Why can channel and spatial attention be used sequentially?**
    They act on different axes, so one does not undo the other, and choosing channels first gives the spatial gate cleaner pooled statistics to work from. In our runs the combination (0.9328) was ahead of channel alone (0.9256) and spatial alone (0.9310), although only by a few test images.
11. **Why might channel attention behave differently at conv3_x and conv5_x?**
    conv3_x channels are generic textures shared by all classes, so re-weighting them carries little class information. conv5_x channels are much more class-selective, but a module there has 527K parameters and sits right before pooling. In our runs this showed up as quick overfitting at P_4 (training accuracy 0.983 by epoch 10, rising validation loss), while P_2 trained slowly and ended lower (0.9238).
12. **Why can spatial attention be useful at an intermediate ResNet stage?**
    At P_3 (14×14) the features are already semantic enough to separate the object from the background, there is still enough resolution to localise, and conv5_x can still learn to use the re-weighted map. P_3 was the only position that beat the baseline.
13. **Why is self-attention computationally expensive when the feature map has high spatial resolution?**
    Every location is compared with every other location, so both memory and computation grow with $N^2 = (HW)^2$. Even at P_3 it was the only module that raised GFLOPs noticeably (7.75 to 8.35).
14. **What happens to the size of the self-attention matrix when the spatial resolution increases?**
    It grows with the fourth power of the side length. Going from P_4 to P_1 takes it from 49×49 (2,401 entries) to 3136×3136 (about 9.8M entries, 37.5 MB per image in float32). For a batch of 32 that would be roughly 1.2 GB for a single attention matrix, which is why it was kept at P_3.
15. **How does attention performance vary when the insertion position is changed?**
    It peaks in the middle: 0.8984 at P_1, 0.9238 at P_2, 0.9328 at P_3 and 0.9256 at P_4. Early modules disturb the inputs of frozen later stages, which mostly cannot adapt, and the late module overfits. Position mattered more than the type of module in this study.
16. **How does attention at P_2 differ visually from attention at P_4?**
    P_2 maps are fine-grained, with thin streaks and texture along the image edges. P_4 maps are coarse 7×7 blobs that damp larger regions, mostly the lower part of the image (stems, leaves). Neither highlights the flower itself.
17. **Does applying attention at multiple stages always improve classification performance?**
    No. $P_3 + P_4$ (0.9111) and $P_2 + P_3 + P_4$ (0.9129) were both worse than P_3 alone (0.9328) and even worse than the baseline, while adding up to 560K more parameters. The $P_3 + P_4$ run also had the highest final validation loss (0.536).
18. **Does Channel→Spatial attention behave differently from Spatial→Channel attention?**
    In final accuracy, hardly (0.9328 vs 0.9310, one image). In behaviour, yes. Spatial→Channel started training very slowly (0.18 validation accuracy after epoch 1), its spatial map had a much narrower range (minimum around 0.5–0.8 vs 0.15–0.2), and it finished with a higher validation loss (0.418 vs 0.331).
19. **How does the spatial-attention kernel size influence the attention map?**
    Larger kernels give smoother maps that depend more on the borders: patchy interior values at $k = 3$, a frame at $k = 5$, and only the left and bottom edges at $k = 7$. Accuracy did not follow this (0.9328, 0.9274, 0.9328), so the extra context did not help here.

20. **How does SE attention differ from ECA attention?**
    SE squeezes the channel descriptor through a two-layer bottleneck (132,160 parameters at P_3). ECA skips the bottleneck and uses a 1D convolution across neighbouring channels (5 parameters). In our runs they were one image apart (0.9256 vs 0.9238), so the bottleneck bought nothing measurable.
21. **How does CBAM combine channel and spatial attention?**
    It first computes channel weights from both average- and max-pooled descriptors passed through a shared MLP, and then applies a 7×7 spatial gate on the channel-refined features. Despite having exactly the same parameter count as our A3 module, it scored 0.9201 against 0.9328. The extra max-pooled branch did not help on this data.
22. **How does self-attention differ from channel attention?**
    Self-attention mixes information between locations through an $N \times N$ matrix; channel attention only rescales channels. In practice, our self-attention collapsed: every query looked at the same one key position, usually on the flower. It therefore behaved like a small global-context term ($\gamma = 0.065$) rather than true pairwise reasoning, at ten times the parameters of channel attention.
23. **Does the attention mechanism with the largest number of parameters necessarily provide the highest accuracy?**
    No. Self-attention (1.31M) was the worst at 0.9002, while the 99-parameter spatial module reached 0.9310 and the 132K-parameter A3 tied for the best. The accuracy–parameters scatter shows no trend.
24. **What is the difference between an internal attention map and Grad-CAM?**
    An internal map is part of the forward computation and directly scales the features, at one layer and for no particular class. Grad-CAM is computed afterwards from gradients of one class score and only describes the decision. In our models they disagreed visibly: the internal spatial maps were near-uniform inside the image, while Grad-CAM for the same model sat squarely on the flower.
25. **Why is Grad-CAM called a post-hoc explanation technique?**
    Because it is computed after the trained model has made its prediction, from that model's activations and gradients, without changing the model or the prediction. That is also why it can be applied to the plain baseline just as easily as to the attention models.
26. **Does the attention model suppress irrelevant background regions?**
    Only partly. The internal maps damp image borders, which sometimes overlap with background (the lower region at P_4), but they do not follow object boundaries. In Grad-CAM, A3 was tighter than the baseline on the tulip-bed scene, yet on the misclassified daisy it was drawn to background highlights. Across the ten images examined, there was no consistent background suppression.
27. **Are the attention maps of correctly classified and incorrectly classified images different?**
    The internal maps are not: the spatial gate gives the same border pattern in both cases. Grad-CAM is different: correct predictions give one compact region on the flower, while the two A3 errors give several scattered blobs on petal edges or background.
28. **Which flower classes benefit most from attention?**
    Sunflowers, by a clear margin (F1 0.9238 to 0.9612), mainly because A3 stopped calling dandelions sunflowers (4 errors to 0). Dandelion improved slightly, daisy was unchanged, and roses actually got worse (0.9005 to 0.8796).
29. **What information can be obtained from a confusion matrix that cannot be obtained from overall accuracy alone?**
    Which classes are mixed up and in which direction. Accuracy only says A3 gained two images; the matrix shows it gained six on dandelion and sunflowers and lost four on tulips and roses, and that tulips→roses remains the main error.
30. **Why is macro F1 useful in addition to accuracy?**
    It gives every class the same weight and balances precision against recall, so a model cannot look good just by doing well on the largest class (dandelion, 135 test images, against 95 for daisy). Here it ranked the models almost exactly like accuracy, which confirms that no model was carried by one class.
31. **What is meant by the accuracy–complexity trade-off?**
    Whether the extra parameters or computation of a module are repaid by accuracy. In this study they mostly were not: the best accuracy was reached with 132K extra parameters, the same score was reached with 1.44M, and a 99-parameter module came within one image.
32. **What is meant by the F1–inference-time trade-off?**
    The same question asked in terms of latency. All models ran at 9.38–9.89 ms per image, so attention costs at most about 5%. The fastest attention variants (ECA, $k = 3$) were as good or nearly as good as the slowest, and the slowest one (self-attention) had the lowest F1.
33. **Why must every ablation study use the same dataset split?**
    Because the effects being measured are tiny. One test image is 0.18%, and most of our differences are two to four images. A different split could easily move the scores by more than that, so the comparison would no longer be about the model.
34. **Why must the same training protocol be maintained when comparing different attention mechanisms?**
    Training choices affect accuracy at least as much as attention does. In our runs, the checkpoint rule alone

changed A4's outcome: it selected an epoch-4 model before fine-tuning. Any difference in epochs, learning rate or augmentation would be mixed into the attention effect and could not be separated from it.

35. **What conclusions can be drawn from the Grad-CAM comparison?**
    The baseline already localises the flower well, and the attention models mostly reproduce its maps with small shifts (A3 more towards petals and buds). Where attention models fail, their maps fragment onto background or edges. The visual evidence therefore agrees with the numbers: attention at P_3 slightly refines, rather than reshapes, what the network relies on.

36. Conclusion

On top of an ImageNet-pretrained ResNet50 that already reached 0.9292 on `tf_flowers`, attention produced small, uneven gains at best. A single Channel→Spatial module after conv4_x was the most reliable choice (0.9328, macro F1 0.9309) for 132K extra parameters and about 0.4 ms per image. Where the module sat mattered more than which module it was: early positions and stacked modules hurt, the late position overfit, and the costly self-attention block collapsed to a single key and gave the lowest accuracy. Lightweight options such as ECA, plain spatial attention and Coordinate attention kept up with the heavier ones.

The visualisations explain why the gains were so modest. The learned spatial gates were nearly uniform inside the image, the channel gates mostly kept a fixed set of channels open, and Grad-CAM changed little from the baseline. All of this comes from a single seed and a 551-image test set, so most differences of a few images should be read as trends rather than firm rankings. Repeating the key runs with several seeds would be the natural next step.

37. References

1. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
2. J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-Excitation Networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 8, pp. 2011–2023, 2020.
3. S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 3–19, 2018.
4. Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11534–11542, 2020.
5. Q. Hou, D. Zhou, and J. Feng, "Coordinate Attention for Efficient Mobile Network Design," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13713–13722, 2021.
6. A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
7. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.
8. TensorFlow Documentation: https://www.tensorflow.org/
9. Keras Documentation: https://keras.io/
10. TensorFlow Datasets: https://www.tensorflow.org/datasets